

CRUCIBLE

An Inference-Native, Self-Improving, Privacy-Preserving Runtime
that Lifts Small Language Models to Frontier-Grade Agency
on a User's Own Distribution

A Research Book on Architecture, Theory, and Engineering

Samuel Dagne

Independent Researcher

Prism Research Press
2026

Copyright © 2026 Samuel Dagne.

Published by Prism Research Press.

All rights reserved. No part of this book may be reproduced, stored, or transmitted without prior written permission, except for brief quotations used in scholarly review, teaching, or criticism.

This manuscript is prepared as a research-book edition for review. Publisher, affiliation, license, ISBN, and edition details should be finalized before external publication.

First review edition: June 5, 2026.

Acknowledgments

The author thanks the researchers and open-source communities whose work on local language models, inference engines, structured decoding, verification, privacy, runtime monitoring, and agent evaluation made this synthesis possible. Any errors, overstatements, or remaining design gaps are the author's responsibility.

Abstract

The dominant assumption of the agent era—that capability is bought with parameters—is, for *agentic* work, largely false. The binding constraint on locally hosted small language models (SLMs, 7–9B) is not scale but *architecture*: prevailing agent frameworks reach the model only through a stateless text API, discarding on every turn the internal control surfaces—the key–value (KV) cache, per-token logit masks, draft-model proposals, process- reward scores, and the ability to change the model—on which the decisive recent advances depend. We call this the *API-boundary wall*.

This treatise develops CRUCIBLE, an *inference-native* runtime that co-locates the agent loop with the inference engine and operates on its internal state, organized into six layers: a Substrate that constrains *emission* but never *cognition* and searches trajectories speculatively; a Cognition layer that allocates compute by its value under a co-evolved verifier; a Memory layer that consolidates while idle; an Evolution layer that improves itself—context, skills, policy, and reversible weights—under a statistically valid gate; a Governance layer with formal runtime guarantees; and an Amplification layer that learns from a frontier teacher and, uniquely, from a privacy-preserving federation of peers. Seven novel mechanisms (EATS, FEATS, VGSS, VoI, STC, ROSE, SGC) are introduced by composing the 2023–2026 literature. We prove the self-improvement loop is non-degrading under adaptive, continual operation, that escalation cost decays geometrically, that federation accelerates that decay under a bounded privacy budget, and we delimit, via the weak-to-strong ceiling, what the system can and cannot promise. CRUCIBLE is both a falsifiable scientific program and a buildable product, and we are explicit about which claims are which.

Preface

This book was written to answer a practical research question: what would an agent runtime look like if it were designed around the inference engine rather than around a stateless text API? The wager is that local small language models do not need to win every open-ended benchmark in order to become powerful agents. They need a runtime that preserves internal state, spends compute selectively, learns from use, keeps private data local, and refuses unsafe self-modification.

The manuscript is therefore both architectural and theoretical. It proposes a stack, mechanisms, proofs, evaluation procedures, and product constraints. Some claims are build claims, some are scientific claims, and some are deliberately stated as falsifiable hypotheses. The reader should treat the book as a blueprint for a research program as much as a description of a system.

Contents

Acknowledgments	v
Abstract	vi
Preface	vii
List of Abbreviations	xv
1 Literature Review and Research Gap	1
1.1 Small Models as Agent Executors	1
1.2 Test-Time Compute and Structured Decoding	1
1.3 Verification, Reward Models, and Runtime Monitors	1
1.4 Memory, Self-Improvement, and Continual Adaptation	1
1.5 Federation and Privacy	2
1.6 Summary of the Gap	2
2 Methodology	3
2.1 Design-Synthesis Method	3
2.2 Formal Method	3
2.3 Engineering Method	3
2.4 Evaluation Method	3
2.5 Validity and Falsification	3
3 Introduction: Context, Problem, and Thesis	5
3.1 Introduction	5
3.1.1 The Wager	5
3.1.2 The Wall	5
3.1.3 Why Now	6
3.1.4 Thesis and Contributions	6
3.1.5 How to Read This Treatise	7
3.2 Architecture, Not Scale: The Evidence	7
3.2.1 Reliability Compounds, and Small Models Bleed It	7
3.2.2 Test-Time Compute is a Parameter Substitute—If Spent Selectively	7
3.2.3 Verification Multiplies Effective Competence	7
3.2.4 Persistence is the One Advantage the Cloud Cannot Copy	8
3.2.5 Synthesis: The Opportunity is Composition Below the Wall	8
3.3 Principles, Threat Model, and Formalization	8
3.3.1 Design Principles	8
3.3.2 Threat Model and Trust Boundaries	9
3.3.3 The Agentic Decision Process	9
3.3.4 Two Kinds of Orchestrator	9
3.3.5 The Central Claim, Made Falsifiable	10

4	The Six-Layer Architecture	11
4.1	Overview of the Stack	11
4.2	Layer 1 — The Substrate	11
4.2.1	Grammar-Scoped Decoding: Emission, Not Cognition	11
4.2.2	Unified Token-and-Action Speculation	12
4.2.3	Atomix Transactional Settlement	13
4.2.4	Verifier-Guided Speculative Search (Preview)	13
4.2.5	Adaptive Compute and the Shared Cache	13
4.3	Layer 2 — Cognition	14
4.3.1	Parallel Decoding for Compute Headroom	14
4.3.2	Activation Steering as a Cheap Capability Dial	14
4.3.3	The Verifier: Generative, Process-Level, and Co-Evolved	14
4.3.4	The Value-of-Information Compute Market (Preview)	14
4.4	The Token Economy: Structured Cognition and Inter-Agent Communication	14
4.4.1	The Cost of Thinking Out Loud	15
4.4.2	Grammar Eliminates Structural Hallucination—Not Factual	15
4.4.3	Compressed Reasoning, Adaptively	16
4.4.4	Inter-Agent Communication Without Natural Language	16
4.4.5	What This Does and Does Not Promise	16
4.5	Layer 3 — Memory and Consolidation	16
4.5.1	Three-Tier Memory	16
4.5.2	Sleep-Time Consolidation	17
4.6	Layer 4 — Evolution	17
4.7	Layer 5 — Governance and Security	18
4.7.1	Capability Sandboxing and Information-Flow Control	18
4.7.2	Temporal-Logic Runtime Monitors	18
4.7.3	Protecting the Self-Improvement Loop	18
4.8	Layer 6 — Amplification and Federation	18
4.8.1	Learned Routing to a Frontier Teacher	18
4.8.2	Federation: Collective Improvement Without Shared Data	18
5	Novel Mechanisms	19
5.1	The Self-Improvement Loop: EATS, SGC, and FEATS	19
5.1.1	Why a Cascade Should Learn	19
5.1.2	The Distillation Operator	19
5.1.3	Teacher-Grounding: The Antidote to Spurious Rewards	19
5.1.4	Self-Generated Curriculum (SGC)	20
5.1.5	The Gate, in Brief	20
5.1.6	Federated EATS (FEATS)	20
5.2	Search, Compute, Consolidation, and Reversible Edits	21
5.2.1	Verifier-Guided Speculative Search (VGSS)	21
5.2.2	The Value-of-Information Compute Market (VOI)	22
5.2.3	Sleep-Time Consolidation (STC)	22
5.2.4	Reversible Orthogonal Self-Edits (ROSE)	22
5.3	Interactions and Complexity of the Seven Mechanisms	23
6	Theory	25
6.1	Geometric Cost Decay	25
6.2	The Gate: Adaptive Non-Degradation	25
6.2.1	Diagnosis of the Naive Gate	25
6.2.2	The Composite Gate	26
6.3	Federated Acceleration and the Weak-to-Strong Ceiling	27

6.4	Efficiency, Assumptions, and Break-Even	28
6.4.1	Standing Assumptions and What Would Falsify Them	28
7	Engineering, Evaluation, and Product	29
7.1	Implementation	29
7.2	Evaluation Protocol	29
7.3	A Worked Example and Cost Economics	30
7.3.1	One Coding Task Through Six Layers	30
7.3.2	Cost Economics with Worked Numbers	31
7.4	Criticisms, Limitations, and Ethics	32
7.4.1	Anticipated Criticisms and Rebuttals	32
7.4.2	Limitations and Threats to Validity	33
7.4.3	Ethics, Safety, and Broader Impact	33
7.5	Adversarial Hardening: Failure Modes and Their Resolutions	33
7.5.1	Fault Isolation: Co-Location Without a Shared Crash Domain	33
7.5.2	Structure Snowballing: A Failure-Recovery Diagnostic Mode	34
7.5.3	Transactional Reality: Reversibility Classes and Micro-VM Snapshots	34
7.5.4	Chain-of-Draft, Bounded	35
7.5.5	Sleep-Time Drift: A Bipartite Reconstruction Gate	35
7.5.6	Holdout Economics: Execution is a Free Oracle	35
7.5.7	ROSE Without Rank Collapse: SLAO and Novelty-Bounded Pressure	36
7.5.8	Federation Without Utility Collapse: FedASK and a Symbolic-First Channel	36
7.5.9	VGSS: Summary-Level Search, an Execution Anchor, and the Search/Gate Firewall	36
7.5.10	Licensing: The Moat Is Downstream, Not the License	37
7.5.11	Net Effect	37
7.6	Adversarial Hardening II: Coherence, Evidence, and the Poisoning Channel	37
7.6.1	Coherence Under Self-Edits: The Frozen/Changing Contradiction	38
7.6.2	Evidentiary Integrity: A Fair Baseline and an Uncontaminated Benchmark	39
7.6.3	The Poisoning Channel: When Learning Ingests the Adversary	39
7.6.4	Safety Under Self-Edit: Orthogonality Is Not a Safety Guarantee	40
7.6.5	Theory Corrections	40
7.6.6	Systems Realism	41
7.6.7	Market Repositioning and a Licensing Adjudication	41
7.6.8	Net Effect and Honest Residuals	41
7.7	Product, Positioning, and Conclusion	42
7.7.1	A New Category, Not a Better Wrapper	42
7.7.2	Product Roadmap	42
7.7.3	Related Systems	42
7.7.4	Conclusion	42
A	Appendices	44
A.1	Notation	44
A.2	The Control-Surface Contract	44
A.3	Extended Algorithms	45
A.4	Attack Catalog	45
A.5	Design Alternatives Considered and Rejected	46
A.6	Glossary of CRUCIBLE Mechanisms	46
	Conclusion and Future Research	47

Publishing Proposal and Peer-Review Plan	48
A.7 Pitch	48
A.8 Target Audience	48
A.9 Market and Contribution	48
A.10 Proposed Peer Review	48
A.11 Contract Materials	48

List of Figures

3.1	The API-boundary wall. Left: prior systems treat the engine as a sealed text service and discard its internal state every turn. Right: CRUCIBLE co-locates orchestrator and engine so that control decisions read logits, fork the cache, see draft and verifier scores, and apply reversible weight edits (Def. 3.2).	6
3.2	Reader map for the review structure. The six former parts are now six chapters; the detailed material lives as sections, subsections, and subsubsections under that stable top-level spine.	7
3.3	Why per-step structural validity must be near-perfect: small errors compound over an episode. Grammar-scoped emission (§4.2.1) pins the top line at 1 by construction.	8
4.1	The CRUCIBLE stack. Novel mechanisms are in small caps; each is developed in Chapter 5. Colors recur in later per-layer diagrams.	12
4.2	Two-phase decoding: cognition is unconstrained; only the emission is grammar-masked, switched on by a trigger tag and released after. Reasoning quality and structural validity are obtained simultaneously rather than traded off.	12
4.3	Indicative token cost. Reasoning compressed by Chain-of-Draft-style drafting falls to $\sim 8\%$ of chain-of-thought at matched accuracy [59]; inter-agent communication via KV-cache/embedding reuse instead of re-verbalized natural language falls by $\sim 2.8\times$ [61, 62]. Both are gated by the verifier so compression never trades away correctness on hard steps.	15
4.4	The day/night split. Cheap, latency-sensitive work runs interactively; expensive, permanent learning is deferred to sleep-time consolidation, where a mistake is caught by the gate and canary before morning.	17
5.1	The EATS closed loop. Labels are teacher-grounded; candidates are generalized by SGC and admitted only through the statistically valid gate (Chapter 6.2), realized as reversible ROSE adapters. The expensive arc runs off the interactive path, in STC.	20
5.2	FEATS. Each instance shares only differentially private, locally gate-passed abstractions; recipients re-gate every incoming abstraction, so poisoned contributions cannot degrade them (Thm. 6.9).	21
5.3	A VGSS step. Numbers are verifier values; the dashed branch is pruned. Width and depth are set by the VOI budget, so search runs deep only where depth pays.	22
6.1	Geometric decay of the escalation rate (Prop. 6.2). A static cascade pays a fixed tax forever; EATS pays a decaying one toward floor δ_0/γ ; FEATS lowers the floor by sharing solved hard cases (Thm. 6.9).	26
6.2	The composite gate. Per-commit validity comes from the DP holdout (Thm. 6.5); long-run acceptance is throttled by online FDR (Prop. 6.8); behavioral safety by governance; and realized harm is bounded by the reversible canary (Prop. 6.7).	26

7.1	Evaluation claim matrix. Component RQs establish local mechanisms; the thesis claim is only accepted on a held-out user distribution against the identical harness wrapped around the frontier baseline.	30
7.2	Fault-isolated co-location: shared-memory state access (zero-copy) with separate crash domains, replacing the single-process reading of Def. 3.2.	33

List of Tables

4.1	The architecture’s risk ledger. Every “high” risk has an explicit guard proved or specified later in the treatise.	13
5.1	When each mechanism runs, its dominant cost, and what bounds it. The interactive path pays only for VOI, occasional VGSS, and rare teacher calls; the expensive, permanent work is deferred to STC.	23
7.1	Evaluation suites and roles. The longitudinal per-domain protocol, not any static leaderboard, is the instrument for the thesis claim.	31
7.2	Red-team scorecard. Each row links to its resolution.	34
7.3	Second red-team scorecard.	38
7.4	CRUCIBLE versus representative systems. The first column is the dividing line. .	42

List of Abbreviations

ACE	Agent context evolution.
API	Application programming interface.
CG-TTRL	Code-grounded test-time reinforcement learning.
DP	Differential privacy.
EATS	Escalation-as-Training-Signal.
FEATS	Federated Escalation-as-Training-Signal.
KV cache	Key-value cache used by transformer inference engines.
LLM	Large language model.
PANDO	Governed reusable agent skill library.
PRM	Process reward model.
ROSE	Reversible Orthogonal Self-Edits.
SGC	Self-Generated Curriculum.
SLM	Small language model.
STC	Sleep-Time Consolidation.
VGSS	Verifier-Guided Speculative Search.
VoI	Value of Information.

Chapter 1

Literature Review and Research Gap

This chapter positions CRUCIBLE against the bodies of work that make the book possible: small-language-model agents, test-time compute, structured decoding, process verification, memory consolidation, self-improvement, runtime governance, routing, and privacy-preserving federation. The purpose is not to repeat every cited paper, but to make the research gap explicit: the literature has produced many powerful control surfaces, while most agent systems still access models through a stateless text API that discards those surfaces.

1.1 Small Models as Agent Executors

The first literature stream argues that small language models can be effective executors for repetitive, specialized, structured agentic tasks when the system surrounding them supplies routing, verification, and persistence. This motivates the local-first premise of the book: the small model is not treated as a stand-alone replacement for a frontier model, but as the resident policy inside a larger runtime.

1.2 Test-Time Compute and Structured Decoding

A second stream shows that capability can be shifted from parameter count to inference-time computation. Parallel sampling, speculative decoding, tree search, finite-state or grammar-constrained decoding, and token-budget-aware reasoning all increase effective competence without changing the base model. The gap is that these techniques are usually exposed as isolated decoding tricks rather than as one runtime substrate for agent control.

1.3 Verification, Reward Models, and Runtime Monitors

A third stream develops verifiers, process reward models, temporal monitors, and safety gates. These works establish that outputs and intermediate steps can be scored, blocked, or rolled back. The unresolved systems question is how to combine these signals continuously, adaptively, and locally without turning the verifier into a new source of reward hacking or validation overfitting.

1.4 Memory, Self-Improvement, and Continual Adaptation

Recent work on context evolution, skill libraries, test-time reinforcement learning, and adapter-based editing suggests that agents can improve with use. The research gap is not whether individual adaptation mechanisms exist; it is whether a local runtime can accept only improvements that pass a statistically valid gate, preserve rollback, and avoid poisoning from the environment it learns from.

1.5 Federation and Privacy

Federated and differentially private learning methods create the possibility of collective improvement without centralizing raw user data. For agent systems, the unresolved challenge is to share only abstractions that are useful, privacy-accounted, and gate-passed. This is the gap addressed by FEATS and the symbolic-first federation channel.

1.6 Summary of the Gap

Across these literatures, the missing artifact is an inference-native agent runtime: one that sees logits, cache state, verifier scores, draft proposals, monitors, memory, and reversible edits as one control plane. The remainder of the book develops that runtime as an architecture, a set of mechanisms, a theory, and an evaluation program.

Chapter 2

Methodology

This book is a constructive systems treatise. Its method is to synthesize recent results into a coherent runtime design, formalize the conditions under which the runtime can improve safely, and specify an evaluation protocol capable of falsifying the central claim.

2.1 Design-Synthesis Method

The design begins from the API-boundary wall and asks which control surfaces are lost when an agent communicates with a model only through text. Each proposed layer is included only when it restores a necessary control surface: constrained emission, shared cache state, speculative search, verification, memory, gated adaptation, runtime governance, or private collective improvement.

2.2 Formal Method

The theory chapters state assumptions, propositions, and theorems about cost decay, non-degradation, federation, compute allocation, and weak-to-strong limits. The formal role of these results is not to prove that every implementation will work; it is to identify the side conditions under which the architecture’s claims are valid and the conditions under which they should fail.

2.3 Engineering Method

The engineering sections map the architecture onto existing implementation artifacts where possible: inference engines, grammar decoders, speculative decoding, RL training paths, sandboxing, and local storage. This keeps the proposal buildable and separates implementation commitments from scientific claims.

2.4 Evaluation Method

The evaluation protocol is pre-registered at the level of research questions, baselines, suites, metrics, and statistical analysis. It separates component claims from the longitudinal thesis claim and requires comparison against a scaffolded frontier model, not an artificially weakened frontier baseline.

2.5 Validity and Falsification

The methodology treats failure as informative. The book predicts that the system should fail on broad, non-stationary, or adversarially poisoned distributions unless specific safeguards hold.

A review-quality test must therefore measure both success on a user's stable distribution and failure outside the assumptions.

Chapter 3

Introduction: Context, Problem, and Thesis

3.1 Introduction

3.1.1 The Wager

A locally hosted 7–9 B model, today, loses to a frontier model on almost any single open-ended prompt. This treatise nonetheless defends a precise and, we will argue, achievable wager: that a *system* built around such a model, running privately on a user’s own machine, can be made to *surpass a frozen frontier model on that user’s own distribution of work*, while costing a fraction as much, leaking none of the user’s data, and improving every week it is used.

The wager is not that small models are secretly as smart as large ones. It is that the gap which matters *for agents* is mostly architectural, and that the architecture which closes it has not been built because everyone is building on the wrong side of a wall.

Key idea. The frontier gap for local agents is mostly a gap of *architecture*, not of *parameters*. Close the architecture gap and a small local model reaches frontier-grade agency on the distribution that matters to its user.

3.1.2 The Wall

Contemporary agent frameworks—cloud orchestrators and the new wave of self-hosted local workspaces alike—share one structural commitment: they reach the model only through a stateless text interface. A prompt string goes in; a completion string comes out; application code parses it and composes the next prompt. Everything the engine knew between those two events is discarded. The KV-cache that holds the model’s working state is dropped and re-prefilled. The logit distribution, in which the model expresses exactly how confident it is and what alternatives it weighed, is collapsed to sampled text. The draft model that could have proposed the next tokens or actions is absent. The process-reward signal that could have told us *which* of several attempts is correct is never computed. And the weights are frozen and untouchable, so nothing learned today survives to tomorrow.

We name this interface the *API-boundary wall*. It is not a minor inefficiency; it forecloses an entire class of techniques the 2024–2026 literature shows to be decisive for small models. Figure 3.1 contrasts the two regimes.

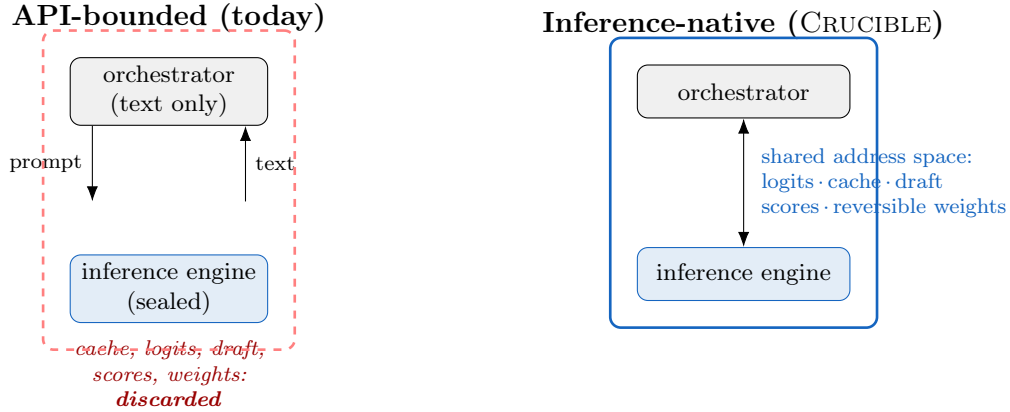


Figure 3.1: The API-boundary wall. Left: prior systems treat the engine as a sealed text service and discard its internal state every turn. Right: CRUCIBLE co-locates orchestrator and engine so that control decisions read logits, fork the cache, see draft and verifier scores, and apply reversible weight edits (Def. 3.2).

3.1.3 Why Now

Three independent lines of evidence have matured to the point where the wager is no longer speculative.

1. **Small models suffice for agentic work.** The case that SLMs should be the *default* executor—reserving frontier calls for the rare hard step—is now explicit, because agentic workloads are dominated by repetitive, specialized, structured sub-tasks [1].
2. **Compute substitutes for parameters.** A 3 B model with compute- optimal test-time scaling matches a 405 B on hard reasoning [2, 3]; constrained decoding alone lets a 3 B beat a 70 B on tool calling [5].
3. **Models can improve themselves—safely, if gated.** Context, skills, policy, and weights can all be adapted on-device [20, 22, 24, 25], and the failure modes (reward hacking, forgetting, validation overfitting) are now well enough understood to be engineered against [26, 28, 29].

What is missing is the runtime that puts these beneath one roof, on the correct side of the wall. That runtime is CRUCIBLE.

3.1.4 Thesis and Contributions

Thesis. *Orchestrating beneath the inference engine—constraining emission but not cognition, speculating and searching over trajectories, spending compute by its value, remembering and consolidating, improving under a statistically valid gate, governed by formal monitors, and improving collectively without sharing data—lets a small local model specialize past a frozen frontier model on its user’s own distribution, and, turned upward, makes the frontier model itself continually better.*

Contributions.

- C1** A formal account of the API-boundary wall and a definition of inference-native orchestration (Chapter 3.3).
- C2** A six-layer reference architecture unifying ~ 60 recent results, with explicit dependency, criticality, and trust maps (Chapter 4).

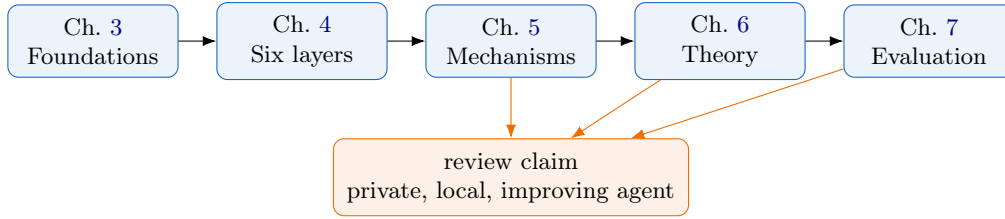


Figure 3.2: Reader map for the review structure. The six former parts are now six chapters; the detailed material lives as sections, subsections, and subsubsections under that stable top-level spine.

- C3** Seven novel mechanisms inferred by composition: EATS, FEATS, VGSS, VoI, STC, ROSE, SGC (Chapter 5).
- C4** A statistically correct theory of continual self-improvement with full proofs (Chapter 6).
- C5** A formal governance layer with runtime guarantees, a comprehensive evaluation protocol, and a product roadmap (Chapter 7).

3.1.5 How to Read This Treatise

Chapter 3 fixes principles, threat model, and formalism. Chapter 4 details the six layers bottom-up. Chapter 5 develops the novel mechanisms. Chapter 6 is the theory. Chapter 7 covers engineering, evaluation, and product. Appendices hold proofs, the control-surface specification, an attack catalog, and a glossary. A reader wanting only the core may read Sections 3.1, 3.3, 5.1–5.3, and Chapter 6.

3.2 Architecture, Not Scale: The Evidence

The architecture is justified only if the ceiling it chases is real. This chapter collects the quantitative case.

3.2.1 Reliability Compounds, and Small Models Bleed It

A model that emits a structurally valid action 95% of the time looks reliable in isolation. Over a twenty-step episode the probability of *at least one* malformed action is $1 - 0.95^{20} \approx 64\%$; at 99% it is still $\approx 18\%$. Production agency needs $\geq 99.9\%$ per step, unreachable by prompting alone [7]. Small models bleed exactly this reliability, and the wall makes it unrecoverable because validation can only happen after generation. Figure 3.3 shows the compounding.

3.2.2 Test-Time Compute is a Parameter Substitute—If Spent Selectively

Compute-optimal scaling lets a 3 B exceed a 405 B on MATH/AIME [2, 3]. But the result is *compute-optimal*: it requires spending more samples on hard steps and none on easy ones, gated by a process verifier. That selective spend needs branch-level control over sampling and cache—a surface the wall hides, and the reason CRUCIBLE introduces a value-of- information market (Chapter 5.2).

3.2.3 Verification Multiplies Effective Competence

Generative verifiers lift best-of- N sharply (GSM8K 73 $\rightarrow$ 93.4%) [14]; process reward models supervise long trajectories [15, 16]. A small generator plus a good verifier behaves like a much

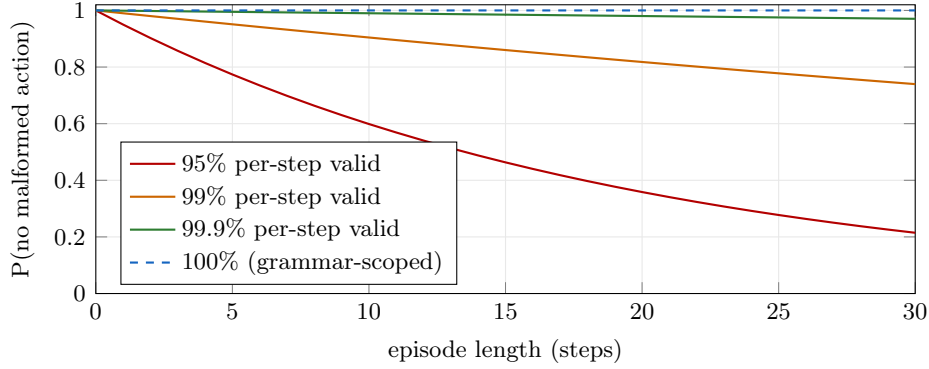


Figure 3.3: Why per-step structural validity must be near-perfect: small errors compound over an episode. Grammar-scoped emission (§4.2.1) pins the top line at 1 by construction.

larger model on verifiable tasks—but only if the verifier can see and score candidate branches inside the loop.

3.2.4 Persistence is the One Advantage the Cloud Cannot Copy

A frozen cloud model is the same model for every user and forgets each between sessions. A local model can keep the user’s data and specialize to it permanently and privately. This is not an omission the cloud could fix; its trust and scaling model *forbids* it. It is therefore the most defensible asymmetry available to a local system—if the runtime can change the model safely (Chapter 4.6).

3.2.5 Synthesis: The Opportunity is Composition Below the Wall

Each mechanism above is individually published, individually decisive, and individually impossible above the wall. The opportunity is their *composition* beneath the wall, plus a loop that turns a frontier teacher into permanent local capability. That composition is the subject of this treatise.

3.3 Principles, Threat Model, and Formalization

3.3.1 Design Principles

The architecture follows from a few commitments, stated so every later decision can be traced to one.

Principle 3.1 (Orchestrate beneath the wall). The agent loop is a function of (context, engine state), not of text alone.

Principle 3.2 (Constrain emission, never cognition). Structure is enforced only on bytes a tool must parse; reasoning is never masked, because masking reasoning degrades it [6].

Principle 3.3 (Spend compute by its value). Compute is allocated, per step, where its expected marginal effect on outcome is largest, as judged by the verifier.

Principle 3.4 (Improve only through a valid gate). No change is committed unless it provably helps on a clean certificate, and every change is reversible.

Principle 3.5 (Ground learning in a teacher, not in itself). The self-improvement signal is a frontier teacher’s verified outcome, not the model’s own vote, because uninformative self-reward invites hacking [26].

Principle 3.6 (Keep data on the device; share only gated abstractions). Raw data never leaves the machine; only differentially private, gated abstractions may be shared.

Principle 3.7 (Make safety a layer, not a prompt). Behavioral safety is enforced by sandboxing and formal runtime monitors, not by natural-language instructions.

3.3.2 Threat Model and Trust Boundaries

Because CRUCIBLE edits its own weights, executes tools, and may federate, its security analysis is part of its design.

Assets. The user’s private data and code; the integrity of the local model’s behavior; the integrity of external resources the agent can affect.

Trust zones. The *trusted core* (the user’s own/LAN machines) runs the policy, verifier, memory, and self-improvement. The *semi-trusted teacher* (a frontier API or larger local model) is trusted for capability but not privacy. The *untrusted environment* (tools, web, servers) may overreach, poison observations, or inject prompts. The *untrusted federation* (other instances, the aggregator) is honest-but-curious at best, malicious at worst.

Non-goals. We do not defend against a compromised core, sub-OS side channels, or a teacher adversarial in capability (we inherit its covered errors, Prop. 6.11).

Consequences. Tools run under capability sandboxes with information-flow control (Chapter 4.7); cross-tenant pooling of raw activations is *prohibited*, since such activations admit prompt-inversion attacks [51, 52]; federated sharing is restricted to DP, gated abstractions (Chapter 4.8).

3.3.3 The Agentic Decision Process

An episode is a finite-horizon, partially observed decision process. At step t the agent holds context c_t . The policy π_θ emits an action token-sequence $a_t \sim \pi_\theta(\cdot \mid c_t, s_t)$ from an action language $\mathcal{L}$ (plans, tool calls, reflections, terminations), where s_t is engine state. The environment applies a_t via transition T (possibly irreversible), returning o_t and updating $c_{t+1} = c_t \oplus a_t \oplus o_t$. The episode ends with outcome reward $R \in [0, 1]$ and cost $C = \sum_t \kappa(a_t)$. We maximize $\mathbb{E}[R]$ under a budget on $\mathbb{E}[C]$ over a user distribution $\mathcal{D}_u$.

3.3.4 Two Kinds of Orchestrator

Definition 3.1 (API-bounded orchestrator). Its sole interface is a map $g : \Sigma^* \rightarrow \Sigma^*$ from prompt to completion, retaining no engine state across calls beyond re-serialized text. The engine state s (cache $\mathcal{K}$, logits z , draft proposals, verifier scores) is hidden and parameters θ are immutable to it.

Definition 3.2 (Inference-native orchestrator). It can: (i) read logits z_t and apply a mask m_t before sampling; (ii) read, fork, and prune the cache tree $\mathcal{K}$; (iii) read draft proposals and process-reward scores; (iv) apply persistent but *reversible* updates $\theta \mapsto \theta'$ between episodes; and (v) schedule idle computation. Its policy is a function of (c_t, s_t) .

Remark 3.3. The mapping from capabilities to layers is exact: grammar-scoped emission and steering need (i); unified speculation, speculative search, and cache sharing need (ii)–(iii); reversible self-edits need (iv); sleep-time consolidation needs (v). None is expressible under Definition 3.1. This is the precise sense in which the wall—not parameter count—is the binding constraint.

3.3.5 The Central Claim, Made Falsifiable

Let θ^{fr} be a frozen frontier model and θ_t the local model after t epochs of self-improvement on $\mathcal{D}_u$; let $\Phi(\cdot)$ be the full CRUCIBLE system around a model. We study

$$\exists t^* : \mathbb{E}_{x \sim \mathcal{D}_u}[R(\Phi(\theta_{t^*}); x)] > \mathbb{E}_{x \sim \mathcal{D}_u}[R(\theta^{\text{fr}}; x)], \quad \mathbb{E}[C(\Phi(\theta_{t^*}))] \ll \mathbb{E}[C(\theta^{\text{fr}})]. \quad (3.1)$$

The claim is restricted to $\mathcal{D}_u$, to the system Φ , and carries a cost side-condition. Chapter 6 characterizes when it holds and proves a ceiling; Chapter 7 operationalizes its test, designed also to detect its *failure* on broad or non-stationary distributions. *Crucially, the honest baseline is the frontier model in the **same** harness, $\Phi(\theta^{\text{fr}})$, not the bare frozen model:* scaffolding dominates raw capability, so comparing a scaffolded local model to an unscaffolded frontier would be rigged. The defensible claim is parity-or-better against a *scaffolded* frontier on $\mathcal{D}_u$ at far lower cost and full privacy; this correction, and the contamination-aware benchmark choice it forces, are developed in §7.6.2.

Chapter 4

The Six-Layer Architecture

4.1 Overview of the Stack

CRUCIBLE co-locates the orchestrator with the inference engine through *zero-copy shared memory* so that orchestration decisions occur within the lifetime of the KV-cache rather than across an API call—but, importantly, in *separate fault domains* rather than one process, so an orchestrator crash cannot destroy the engine or its cache (the systems rationale and design are in §7.5.1). We build on SGLang for its co-located structured generation and RadixAttention prefix sharing [11, 5], and on a veRL/HybridFlow path for reinforcement-learning components [12]. The runtime has six layers (Figure 4.1) in a strict dependency order: each layer presupposes the control surfaces beneath it, and the API-boundary wall of prior systems sits *above* the whole stack.

4.2 Layer 1 — The Substrate

The Substrate is the set of capabilities that exist only beneath the wall. Its job is to make the small model *structurally incapable of failing* and *fast enough to think more*, without touching what the model reasons about.

4.2.1 Grammar-Scoped Decoding: Emission, Not Cognition

The naive temptation—and a real failure of CRUCIBLE’s earlier drafts—is to constrain the entire trajectory to a grammar. This is wrong. Forcing a model to *reason inside* a rigid schema measurably degrades reasoning, while constraining only the structured *emission* does not, and constraint actually *helps* on selection tasks [6]. The mechanism of the harm is concrete: constrained decoding masks the logits, so if the model’s natural continuation is a reasoning token that momentarily violates the schema, the decoder forces a lower-probability alternative and derails the chain.

CRUCIBLE therefore adopts a two-phase policy (Figure 4.2). In the *cognition phase* the model generates entirely unconstrained, preserving its pretraining distribution. On a trigger tag (XGrammar-2 TagDispatch [5]) the runtime activates the relevant tool-schema grammar for the *emission only*, then releases. A small trajectory grammar G_{traj} governs only the legal *sequence of phases*, never the content of reasoning:

$$\langle \text{episode} \rangle ::= (\langle \text{think} \rangle (\langle \text{emit} \rangle \mid \langle \text{reflect} \rangle))^* \langle \text{halt} \rangle, \quad \langle \text{think} \rangle \rightarrow \Sigma^* (\text{free}), \quad \langle \text{emit} \rangle \rightarrow \text{schema}.$$

This removes the malformed-action failure mode *by construction*—the line Figure 3.3 pins at 1—while leaving cognition intact. Where even the emission benefits from drafting first, a fallback *NL-to-Format* pass lets the model reason free, then translate to schema in a second constrained pass [6].

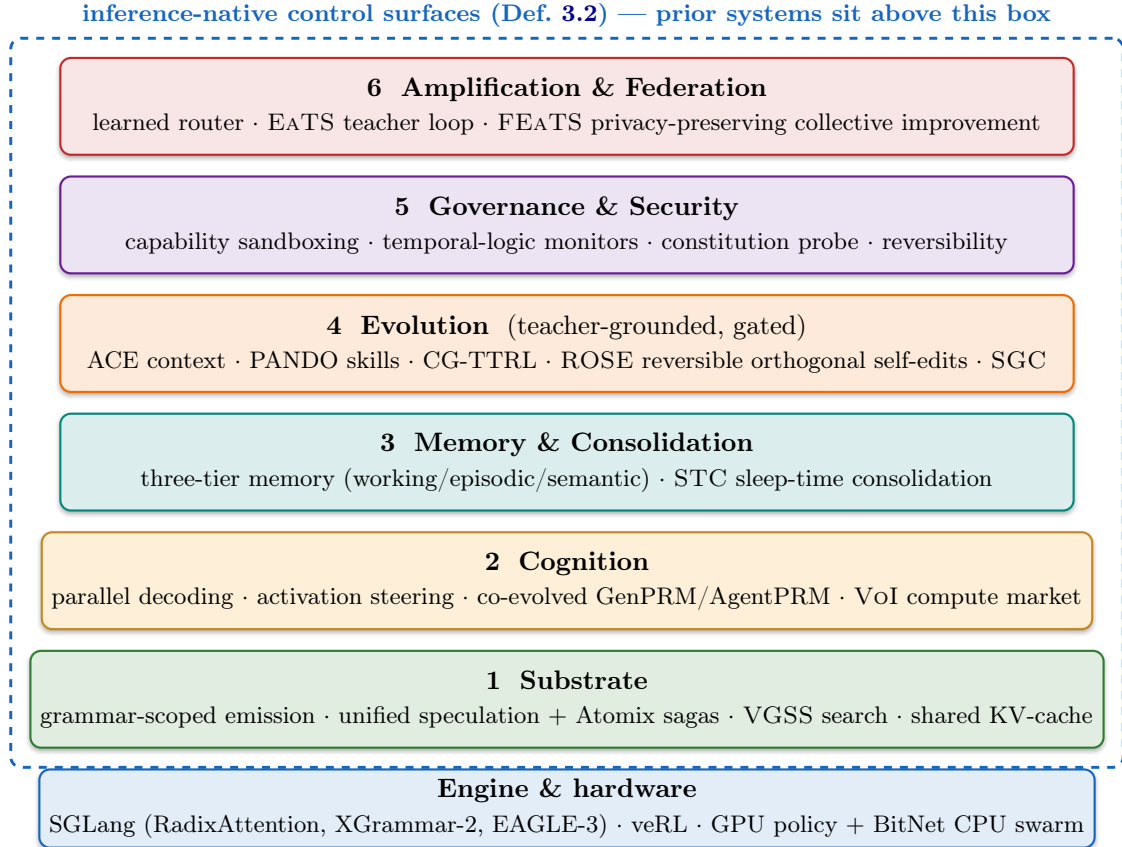


Figure 4.1: The CRUCIBLE stack. Novel mechanisms are in small caps; each is developed in Chapter 5. Colors recur in later per-layer diagrams.

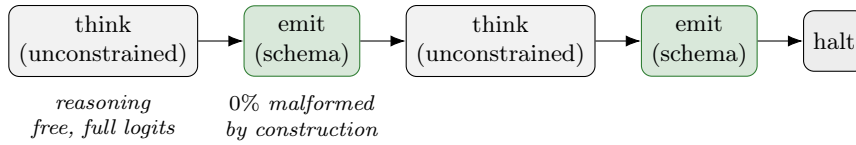


Figure 4.2: Two-phase decoding: cognition is unconstrained; only the emission is grammar-masked, switched on by a trigger tag and released after. Reasoning quality and structural validity are obtained simultaneously rather than traded off.

4.2.2 Unified Token-and-Action Speculation

Latency on local hardware is dominated by autoregressive decoding and by waiting on tools; CRUCIBLE attacks both with one mechanism. An EAGLE-3-style self-speculation head—a lightweight autoregressive head conditioned on early, middle, and late hidden states of the target—drafts tokens the target verifies in a single pass, yielding $2\text{--}6\times$ throughput at $0.75\text{--}0.85$ acceptance without changing outputs [8]; a small or ternary BitNet draft model can serve the same role [34]. The *same* draft mechanism proposes next *actions*: side-effect-free actions (reads, searches) are pre-executed during the target’s think time and their results placed in cache, while side-effecting actions are staged (§4.2.3). Token- and action-level speculation share one cache tree, so a rejected speculation discards a subtree rather than recomputing, and an accepted one is already warm. This unification—one draft model, one cache, both levels—is, to our knowledge, not present in existing runtimes, which treat token speculation (a serving optimization) and action speculation (an agent optimization) as separate systems with separate state.

Component	Necessity	Risk	Primary guard
Grammar-scoped emission	necessary	low	emission-only scope; free cognition
Unified speculation	high value	medium	Atomix transactional settlement
VGSS search	high value	medium	verifier pruning; side-effect-free only
Co-evolved verifier	necessary	medium	adversarial hardening; isomorphic checks
VOI compute market	high value	low	verifier-estimated value; budget cap
Memory + STC	high value	low	bounded stores; gated consolidation
ACE / PANDO	high value	low	gate + confidence demotion
CG-TTRL / ROSE	high value	high	DP gate + FDR + orthogonal LoRA + rollback
Governance monitors	necessary	low	formal LTL guarantees; sandbox
Learned routing	high value	low	calibrated thresholds; transfer
EATS / FEATS	high value	high	teacher-grounded; DP federation
Latent reasoning	excluded	—	negative result [29]

Table 4.1: The architecture’s risk ledger. Every “high” risk has an explicit guard proved or specified later in the treatise.

4.2.3 Atomix Transactional Settlement

Naive “execute then roll back” is unsafe under concurrency and impossible for irreversible external effects. CRUCIBLE settles speculative actions through the Atomix model [9]: *epochs* (logical timestamps imposing a total order), *scopes* (typed read/write resources), *effects* (structured side-effect descriptions with idempotency keys and compensation handlers), *frontiers* (per-resource monotonic commit cursors), and *transactions* (groups that settle or abort atomically). An effect becomes permanent only when its resource frontier passes its epoch, certifying no earlier-epoch work can still arrive; microbenchmarks report success under heavy fault injection rising from 0–7% (immediate execution) to 37–57% with this discipline [9]. Each tool adapter carries a *capability schema* (Chapter 4.7) bounding the scopes it may touch, closing the capability-overreach gap unconstrained tool protocols leave open.

4.2.4 Verifier-Guided Speculative Search (Preview)

Speculation drafts one likely continuation; *search* considers several and keeps the best. CRUCIBLE generalizes single-line speculation into a bounded tree search over trajectories—VGSS—in which branches are proposed by the draft model, scored and pruned by the process verifier, and (for side-effect-free branches) pre-executed transactionally, all sharing one KV-cache. VGSS fuses the MCTS-style search of LATs [10] with speculative decoding and process verification; it is developed in Chapter 5.2.

4.2.5 Adaptive Compute and the Shared Cache

Each step is assigned a compute level by the VOI controller (Chapter 5.2); higher levels fork more branches off the shared prefix, cheap because RadixAttention shares the common KV-cache

across branches [11]. The cache tree is a first-class object the orchestrator reads and prunes—the capability that makes both VGSS and best-of- N affordable on one consumer GPU.

4.3 Layer 2 — Cognition

The Cognition layer governs *how* the model thinks, and how hard, given the substrate beneath it.

4.3.1 Parallel Decoding for Compute Headroom

Where the model or a draft supports it, parallel/diffusion decoding raises tokens per second by up to an order of magnitude [30, 31]. The point is not raw speed but *headroom*: at a fixed latency budget, faster decoding buys more test-time compute, the resource that lets small models close reasoning gaps (Chapter 3.2). Parallel decoding is thus an enabler of Layer-2 adaptivity, not an end in itself.

4.3.2 Activation Steering as a Cheap Capability Dial

Reasoning behaviors—verification, backtracking, caution—correspond to approximately linear directions in activation space that can be added at inference time [32, 33]. CRUCIBLE keeps a small bank of such control vectors and a lightweight policy that chooses which to apply, and at what gain, per step. Steering is a strictly cheaper alternative to best-of- N for nudging a hard step: one steered pass rather than N samples. It is held off the critical path (ablatable) because its reliability is model-dependent.

4.3.3 The Verifier: Generative, Process-Level, and Co-Evolved

The verifier V_ϕ is the most load-bearing component in the system: it selects speculative branches, prunes the VGSS tree, sets the compute level, and gates self-edits. CRUCIBLE uses a generative process-reward model (GenPRM)—which performs explicit derivation and, where possible, code execution before scoring a step [15]—augmented with AgentPRM turn-level value functions learned via temporal-difference and generalized advantage estimation for long trajectories [16]. Because a frozen judge is eventually gamed by an improving policy, the verifier is *co-evolved*: trained against the policy’s evolving outputs with adversarial hardening and, wherever feasible, grounded in execution (isomorphic verification), which eliminates reward-hacking gaps that purely extensional checks induce [17, 18, 19]. The verifier’s quality is treated as a first-class, measured quantity (Chapter 7.2), because the system’s safety reduces to it.

4.3.4 The Value-of-Information Compute Market (Preview)

Compute on one machine is genuinely scarce, and uniform spending is wasteful: most agent steps are trivial and a few are pivotal. CRUCIBLE allocates compute—branches, steering, escalation—by a value-of-information criterion estimated from the verifier, treating the GPU and BitNet swarm as a small market that clears each step. The mechanism is developed in Chapter 5.2.

4.4 The Token Economy: Structured Cognition and Inter-Agent Communication

This chapter is cross-cutting: it draws on the Substrate’s grammar and shared cache (Chapter 4.2) and the Cognition layer’s verifier (Chapter 4.3) to attack what is, on both cloud and local hardware, the dominant cost of agency—*tokens*—and the dominant reliability failure—*hallucinated structure*. It is also, as Chapter 7 argues, the most natural first product, because its benefits are immediate and measurable without any of the heavier self-improvement machinery.

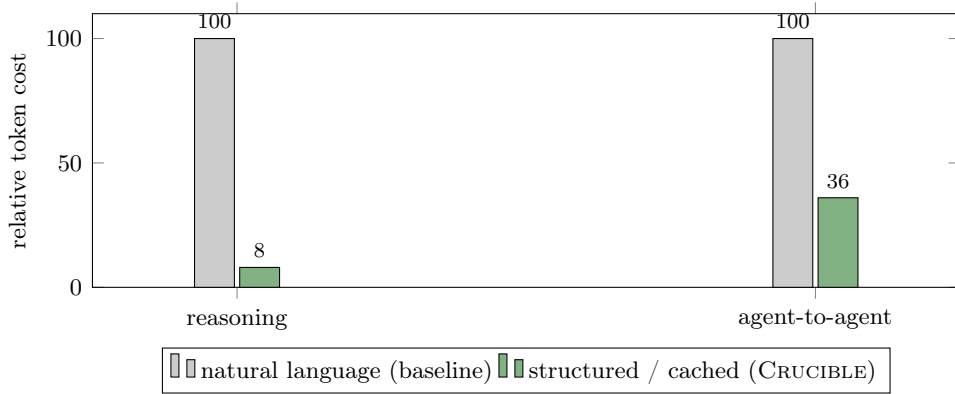


Figure 4.3: Indicative token cost. Reasoning compressed by Chain-of-Draft-style drafting falls to $\sim 8\%$ of chain-of-thought at matched accuracy [59]; inter-agent communication via KV-cache/embedding reuse instead of re-verbalized natural language falls by $\sim 2.8\times$ [61, 62]. Both are gated by the verifier so compression never trades away correctness on hard steps.

4.4.1 The Cost of Thinking Out Loud

Modern agentic workflows are extravagant with tokens. A single agentic task—retrieve context, reason through steps, call tools, validate—commonly consumes 15,000–80,000 tokens, against 500–2,000 for a simple question. Two sources dominate: *reasoning*, where chains of thought run to thousands of redundant “thinking” tokens, and *inter-agent communication*, where every turn of a multi-agent exchange re-encodes the entire accumulated conversation in natural language—a four-agent, five-round debate is twenty full model calls at minimum. On a metered cloud this is a budget crisis; on local hardware it is a *latency and throughput* crisis with the same shape. CRUCIBLE treats token economy as a first-class objective, not an afterthought.

4.4.2 Grammar Eliminates Structural Hallucination—Not Factual

The single most reliable token-and-error saving is to make malformed output impossible. Finite-state/grammar-constrained decoding over the schema of *valid tool names*, *arities*, and *argument types* reduces structural errors to zero: methods of the ToolDec family report 0% syntax errors across benchmarks, eliminating name, arity, type, and structure mistakes [64, 5]. The agent becomes *incapable* of calling a non-existent tool, mismatching arguments, or emitting unparseable JSON—removing both the failure and the wasted tokens of the retry loop (recall the compounding arithmetic of Figure 3.3).

Key idea. Grammar-constrained *emission* makes invalid *moves* impossible (zero structural hallucination), but a syntactically perfect call can still carry an *invented argument value*. Structure is solved by the grammar; *facts* are solved by the verifier and retrieval. CRUCIBLE uses both, and is careful never to claim the first solves the second.

This is the honest boundary of the “near-zero hallucination” claim. A constrained decoder guarantees a well-formed call; it does not guarantee a *true* one [64]. Factual/semantic hallucination—a plausible but wrong value, a mis-grounded fact—is handled elsewhere in CRUCIBLE: by the co-evolved process verifier (§4.3.3), by execution-grounded (isomorphic) checks where available, and by retrieval from semantic memory. The division of labor is the point: grammar for structure, verification for truth.

4.4.3 Compressed Reasoning, Adaptively

Most “thinking” tokens are redundant. Chain-of-Draft constrains each reasoning step to a terse, ~ 5 -word sketch and matches chain-of-thought accuracy at 7.6% *of the tokens*, with 48–76% lower latency [59]; token-budget-aware reasoning sets a per-problem thinking budget [60]; sketch-of-thought generalizes the idea. CRUCIBLE does not apply these uniformly, for two reasons grounded in the evidence: forcing reasoning into a rigid form can hurt [6, 65], and concise reasoning transfers unevenly to hard code tasks [59]. Instead, the VoI controller (§5.2.2) sets the *reasoning budget per step*: trivial steps get a one-line draft, pivotal steps get full free-text reasoning or a VGSS expansion, and the verifier checks that compression did not cost correctness. Token economy thus becomes another dimension of value-of-information allocation, not a blanket constraint.

4.4.4 Inter-Agent Communication Without Natural Language

The most wasteful tokens in multi-agent systems are spent re-saying in prose what the sender already computed. Because CRUCIBLE co-locates agents in one runtime over a shared KV-cache (Chapter 4.1), it can replace natural-language hand-offs with three cheaper channels: (i) *typed message schemas*—a grammar over inter-agent messages, so a hand-off is a compact, parseable structure rather than a paragraph, with the same zero-malformed guarantee as tool emission; (ii) *KV-cache reuse*—a receiving agent reuses the sender’s cached context instead of re-prefilling it, in the DroidSpeak/KVComm manner, for up to $2.78\times$ lower prefill cost at negligible accuracy loss [61, 62]; and (iii) *embedding-space messages*, in the CIPHER manner, where agents exchange compact latent summaries rather than tokens [63]. These are available to CRUCIBLE precisely because it lives below the wall—an API-bounded orchestrator cannot share a cache or an embedding between calls. Sparse-communication results further bound how often agents need to talk at all, beyond which returns diminish.

4.4.5 What This Does and Does Not Promise

Collecting the honest accounting: CRUCIBLE drives *structural* hallucination to ≈ 0 by construction (§4.4.2); it reduces *reasoning* tokens by roughly an order of magnitude on amenable steps and *inter-agent* tokens by a small integer factor (§§4.4.3–4.4.4), verifier-gated so accuracy is preserved; and it pushes residual *factual* hallucination onto the verifier and retrieval rather than the grammar. It does *not* reach literally zero tokens (answers and actions must still be emitted) nor literally zero hallucination (truth is not a grammar). The defensible, and still remarkable, claim is: *a local agent that cannot make a malformed or invalid call, and that spends a fraction of the tokens of a natural-language agent at matched accuracy*—which, given the token-cost crisis of agentic workflows, is on its own a compelling product (Chapter 7).

4.5 Layer 3 — Memory and Consolidation

A persistent local agent accumulates experience; whether that becomes capability or clutter depends on the memory architecture.

4.5.1 Three-Tier Memory

Following the convergent 2025–2026 design and its cognitive-science lineage, CRUCIBLE maintains three tiers [36, 37]: *working memory* (always in context, the current task’s state), *episodic memory* (a bounded log of past trajectories and outcomes), and *semantic memory* (distilled, deduplicated knowledge and the ACE playbook). Paging between tiers is under the agent’s control via tool calls, but—unlike systems that consolidate only on demand—CRUCIBLE runs an

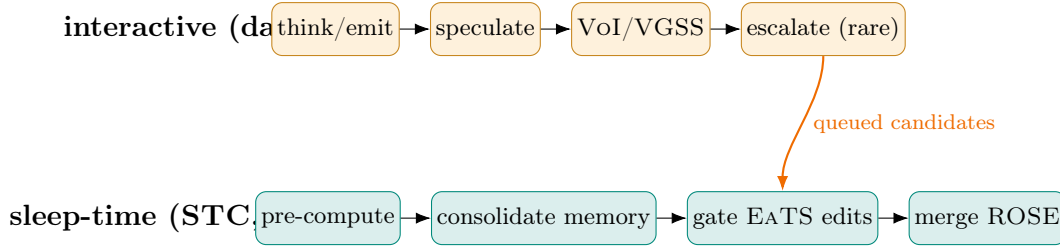


Figure 4.4: The day/night split. Cheap, latency-sensitive work runs interactively; expensive, permanent learning is deferred to sleep-time consolidation, where a mistake is caught by the gate and canary before morning.

automatic consolidation pipeline (STC), which prevents episodic memory from growing without bound and keeps retrieval sharp.

4.5.2 Sleep-Time Consolidation

Idle hardware is unrealized capability. During idle or charging windows CRUCIBLE runs STC (Figure 4.4): it performs sleep-time compute over the day’s contexts—pre-digesting and rewriting them into reusable inferences before any future query [38]—consolidates episodic traces into semantic memory, and triggers the gated EATS distillation and adapter merges of Layer 4. STC is the operational realization of “the agent gets better while you sleep,” and the reason the interactive path stays fast: the system’s most expensive, most permanent work is deliberately moved off it. The mechanism is detailed in Chapter 5.1.

4.6 Layer 4 — Evolution

Evolution is self-improvement at four granularities of increasing permanence and risk, all driven by a single teacher-grounded signal and all subject to the gate of Chapter 6.

Context (ACE). The lowest-risk tier evolves an itemized playbook of strategies and failure modes via delta updates merged by non-LLM logic, avoiding context collapse and improving agents without labels [20].

Skills (PANDO governance). Reusable code skills are accumulated as in Voyager [21] but governed by the PANDO lifecycle [22]: deterministic keyword retrieval (auditable, cache-friendly, drift-free), polarity-pair merging of matched success/failure trajectories into a single boundary-aware routine, and confidence demotion that blacklists chronically failing skills—higher success at 58% fewer tokens, with step-overhead falling an order of magnitude.

Policy (CG-TTRL). On-device reinforcement learning grounds pseudo-labels in retrieved context and, in the core loop, in teacher-verified escalations rather than bare majority votes [24, 23]—the choice that neutralizes the spurious-reward failure mode (§5.1.3).

Weights (ROSE). The highest-risk tier edits weights via SEAL-style self-generated data [25], but applied as *reversible orthogonal low-rank adapters* so edits cannot catastrophically forget and can be dropped or merged instantly (Chapter 5.2 develops ROSE).

4.7 Layer 5 — Governance and Security

Behavioral safety in CRUCIBLE is a layer with formal guarantees, not a paragraph of instructions the model may ignore; it also defends the self-improvement loop, itself an attack surface.

4.7.1 Capability Sandboxing and Information-Flow Control

Every tool runs under least privilege in an isolated environment (microVM- or gVisor-class) with no ambient network or filesystem authority, consistent with current sandboxing practice and the guidance that agent-generated code never execute without strict sandboxing and allowlisting. Each adapter declares a *capability schema*—the typed scopes (§4.2.3) it may read and write—and the runtime enforces information-flow constraints, so a tool granted read-only file access cannot exfiltrate via a network effect.

4.7.2 Temporal-Logic Runtime Monitors

Natural-language guardrails offer no guarantee. CRUCIBLE compiles safety requirements into temporal-logic specifications—“authenticate before accessing data,” “never write outside the workspace,” “confirm before an irreversible effect”—and enforces them with a runtime monitor that *anticipates* violations and blocks the offending action before it settles [43, 44, 45]. Because the monitor sits at the emission/settlement boundary (Layer 1), it has the action and its declared effects in hand before they become permanent—genuine pre-execution guarantees, not post-hoc detection.

4.7.3 Protecting the Self-Improvement Loop

The gate of Chapter 6 protects against *statistical* degradation; governance protects against *behavioral* regression. A frozen *constitution probe*—a held-out battery of safety behaviors—is in the canary set, so any self-edit degrading a safety behavior is rejected by the same reversible mechanism that rejects competence regressions. Self-edit and escalation logs are immutable and auditable; federated updates are admitted only after passing local monitors.

4.8 Layer 6 — Amplification and Federation

The top layer is where a small local model reaches beyond itself—to a frontier teacher and to a community—without surrendering its advantages.

4.8.1 Learned Routing to a Frontier Teacher

A frontier model θ^{fr} serves two roles. As an *escalation tier* it is invoked only when a learned, preference-data router predicts the local system cannot reach parity, escalating against a calibrated cost–quality threshold and transferring across model swaps [46, 47]. As a *teacher* its escalated trajectories are the raw material for EATS (Chapter 5.1). The same six layers can wrap around θ^{fr} itself, yielding a continually specializing frontier model.

4.8.2 Federation: Collective Improvement Without Shared Data

The most consequential capability of CRUCIBLE is that a community of instances can improve *together* while each keeps its data entirely local, sharing only differentially private, gate-passed abstractions—skills, playbook deltas, and orthogonal adapter updates—never raw data or activations [48, 49, 50]. This is a network effect the cloud’s privacy posture cannot match in reverse; it is developed, with its privacy accounting, in Chapter 5.1. Cross-tenant pooling of raw activations is *prohibited* (§3.3.2).

Chapter 5

Novel Mechanisms

Chapter 4 placed components in layers. This chapter develops the seven mechanisms new to CRUCIBLE—each obtained by composing published ideas across the literature in a way that, to our knowledge, has not been built—giving each its intuition, its precise operation, an algorithm, and a forward reference to its guarantee in Chapter 6.

5.1 The Self-Improvement Loop: EATS, SGC, and FEATS

5.1.1 Why a Cascade Should Learn

A static cascade routes every recurrence of a hard case upward and pays the frontier cost each time [47]. This is a memoryless policy over a distribution with structure: the hard cases a user meets recur and cluster. *Escalation-as- Training-Signal* (EATS) converts each escalation into a labeled example from the teacher and distills it into permanent local capability, so the next instance is handled locally (Figure 5.1). Over a sufficiently stationary $\mathcal{D}_u$ the escalation rate, and hence cost, decays geometrically toward an irreducible floor (Prop. 6.2).

5.1.2 The Distillation Operator

When step or episode x is escalated and the teacher returns a verified trajectory τ^{fr} , EATS forms candidate updates at all four Evolution granularities, $\Delta(x, \tau^{\text{fr}}) = \{\delta_{\text{ctx}}, \delta_{\text{skill}}, \delta_{\text{rl}}, \delta_{\text{wt}}\}$: an ACE playbook bullet; a PANDO skill if τ^{fr} contains a generalizable procedure; a CG-TTRL replay item; and a ROSE self-edit. Before any is gated, SGC (§5.1.4) expands x into a curriculum of variations so that what is learned generalizes beyond the single escalated instance.

5.1.3 Teacher-Grounding: The Antidote to Spurious Rewards

A central safety property of EATS is structural. The training signal is the teacher’s *verified outcome*—an external, informative label—not the model’s own majority vote. This matters because the spurious-reward phenomenon (even random rewards can “improve” a model) is specific to *uninformative* reward and is model-dependent, amplifying a pretraining prior: random rewards lift Qwen2.5-Math but fail on Llama3 and OLMo2 [26, 27]. Grounding in a teacher and verifying with a co-evolved process model places EATS in the regime where the signal is informative. The empirical claim that such loops *must* collapse is further contradicted by the Reward Hacking Benchmark, on which exploit rates span 0–13.9% and simple environmental hardening cuts hacking 87.7% [28].

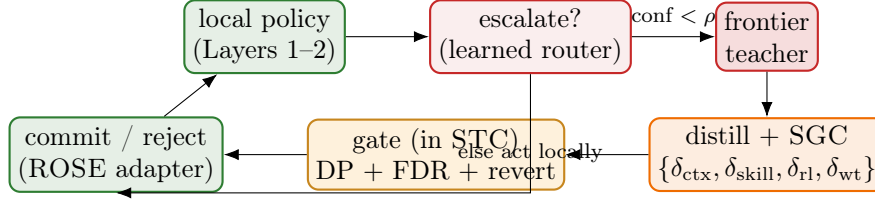


Figure 5.1: The EATS closed loop. Labels are teacher-grounded; candidates are generalized by SGC and admitted only through the statistically valid gate (Chapter 6.2), realized as reversible ROSE adapters. The expensive arc runs off the interactive path, in STC.

Algorithm 1 EATS: one episode with deferred, gated distillation

Require: local θ , teacher θ^{fr} , verifier V_ϕ , router Ψ , threshold ρ

- 1: $c \leftarrow$ context from working memory and retrieved skills/playbook
 - 2: **while** not halted **do**
 - 3: **think** (free) then, on tag, **emit** (schema-constrained)
 - 4: propose tokens/actions (draft); verify on target; settle via Atomix
 - 5: $b \leftarrow \text{VOL.LEVEL}(c)$; explore VGSS tree to budget b
 - 6: $a^* \leftarrow$ best verified action; $\text{conf} \leftarrow \Psi(c)$
 - 7: **if** $\text{conf} < \rho$ **then**
 - 8: $\tau^{\text{fr}} \leftarrow$ teacher solves sub-problem; $a^* \leftarrow$ its action
 - 9: enqueue $\text{SGC}(\Delta(x, \tau^{\text{fr}}))$ for the offline gate
 - 10: **end if**
 - 11: execute a^* under governance monitors; update c , episodic memory
 - 12: **end while**
 - 13: **return** outcome; queued candidates are gated during the next STC window
-

5.1.4 Self-Generated Curriculum (SGC)

A single escalated example teaches narrowly and invites memorization. Before gating, SGC expands each escalated case x into a small curriculum of semantically equivalent and adversarial variations—paraphrases, perturbed inputs, harder near-neighbors—generated by the model under the teacher’s verified solution, in the spirit of Voyager’s automatic curriculum and SEAL’s self-generated data [21, 25]. The candidate edit must improve competence on the *held-out* variations, not merely on x , to pass the gate. SGC is what turns EATS from rote case-memorization into genuine generalization, and makes the holdout certificate (Chapter 6.2) a test of transfer rather than of fit.

5.1.5 The Gate, in Brief

A candidate δ is committed only if it passes a composite test: a differentially private reusable holdout certifies a competence margin valid under adaptive reuse [53]; online false-discovery-rate control throttles acceptance over the unbounded update sequence [55]; and a reversible checkpoint with a rolling canary bounds realized harm. The full construction and theorem are in Chapter 6.2; the episode loop is Algorithm 1.

5.1.6 Federated EATS (FEATS)

A network effect without a data grab. The cloud’s advantage is aggregate data; its constraint is that users must hand it over. FEATS captures the network effect while inverting the constraint: a community of instances improves *collectively* while each keeps its data strictly local, sharing only *differentially private, gate-passed abstractions*—ACE deltas, PANDO skills,

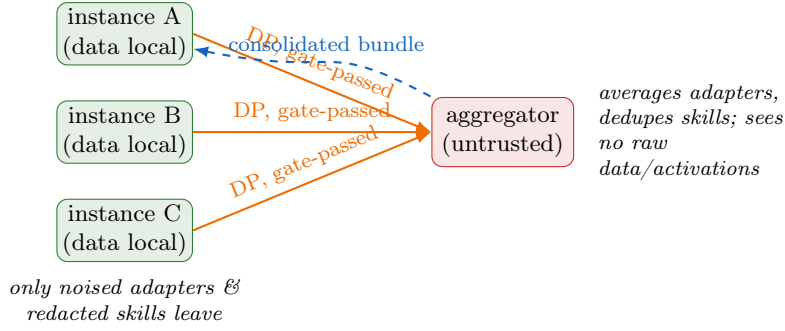


Figure 5.2: FEATS. Each instance shares only differentially private, locally gate-passed abstractions; recipients re-gate every incoming abstraction, so poisoned contributions cannot degrade them (Thm. 6.9).

and ROSE adapter updates—never raw data, contexts, or activations (Figure 5.2).

Protocol. Each round (typically during STC): (i) an instance computes candidate abstractions locally and passes them through its own gate and governance monitors; (ii) it adds calibrated noise to numeric components and redacts/hashes symbolic components under a differential-privacy budget [50, 49, 48]; (iii) it uploads only these to an aggregator, which averages compatible adapters and deduplicates skills and redistributes a consolidated bundle; (iv) each recipient admits an incoming abstraction *only* if it passes that recipient’s *own* local DP-holdout gate and monitors. Step (iv) is essential: federation never bypasses the local gate, so a poisoned or merely unhelpful contribution cannot degrade a recipient (Thm. 6.9 applies unchanged), giving Byzantine robustness by construction rather than by trust.

Why this is the moat. FEATS produces compounding returns that scale with community size while preserving each user’s privacy—an asymmetry the cloud cannot answer in reverse. It accelerates every instance’s cost-decay curve (Chapter 6.1): a rare case solved by *anyone* becomes locally cheap for *everyone*, lowering the effective novelty rate δ_0 in Proposition 6.2. The privacy cost is bounded and accounted in Theorem 6.9.

5.2 Search, Compute, Consolidation, and Reversible Edits

5.2.1 Verifier-Guided Speculative Search (VGSS)

From a line to a tree. Speculative decoding commits to one likely continuation; for pivotal steps a single line is fragile. VGSS replaces it with a bounded tree search over trajectories (Figure 5.3), fusing three ideas not previously combined: the MCTS-style language-agent search of LATS, which uses the LLM as agent, value, and optimizer with environment feedback [10]; speculative decoding, which makes each node cheap; and the co-evolved process verifier, which supplies node values without ground truth.

Operation. At a step selected by VoI, VGSS runs bounded selection–expansion–evaluation–backup: (1) *select* by upper-confidence on verifier value; (2) *expand*—the draft model proposes several next actions in parallel, sharing the parent’s KV-cache prefix, so expansion is near-free [11]; (3) *evaluate*—the GenPRM/AgentPRM verifier scores each child, pre-executing side-effect-free children transactionally to sharpen estimates while scoring side-effecting ones without execution [9, 15, 16]; (4) *backup*—values propagate to the root, the best verified action is emitted, and the rest of the tree is discarded (cache subtrees freed). VGSS subsumes best-of- N (a depth-1 tree)

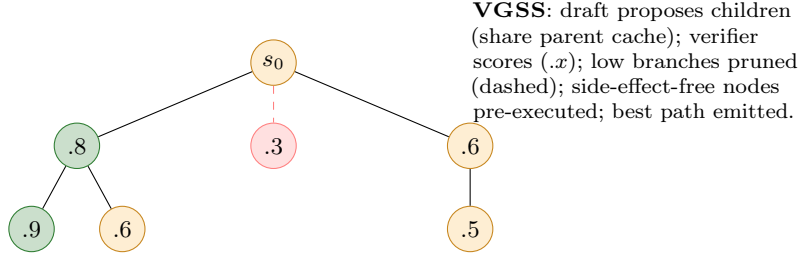


Figure 5.3: A VGSS step. Numbers are verifier values; the dashed branch is pruned. Width and depth are set by the VoI budget, so search runs deep only where depth pays.

and single speculation (a width-1 line) as special cases and inherits LATS’s search behavior at a fraction of its cost through cache sharing and verifier pruning.

5.2.2 The Value-of-Information Compute Market (VoI)

Compute is scarce and lumpy. On one consumer machine, FLOPs are limited and the value of an extra sample is wildly non-uniform: trivial steps need none, pivotal steps deserve much, and a rare step deserves the frontier. Uniform or threshold spending wastes the budget. VoI allocates compute by expected value of information.

Mechanism. At step t with context c , the verifier estimates, for each available compute action u (an extra branch, a steering pass, a deeper VGSS expansion, or an escalation), the expected improvement $\Delta\hat{V}(u | c)$ and the cost $\kappa(u)$ across the GPU and BitNet swarm. Under a per-episode budget B , VoI spends the next marginal unit where value-per-cost is largest, stopping when it falls below a shadow price λ_t :

$$u^* = \arg \max_u \frac{\Delta\hat{V}(u | c)}{\kappa(u)}, \quad \text{spend while } \frac{\Delta\hat{V}(u^* | c)}{\kappa(u^*)} \geq \lambda_t,$$

with λ_t updated online to keep expected spend within B . The swarm thus clears as a small market each step: cheap ternary verifiers and draft heads bid low and run often; the frontier bids high and runs rarely. This makes “adaptive test-time compute”—the mechanism by which small models reach frontier reasoning [3, 2]—an explicit, auditable budget rather than a heuristic. Its near-optimality and noisy-estimate regret are proved in Chapter 6.4.

5.2.3 Sleep-Time Consolidation (STC)

Idle hardware is unrealized capability. During idle or charging windows STC performs, in order: (1) *sleep-time compute*, pre-computing and rewriting the day’s contexts into reusable, query-ready inferences [38]; (2) *memory consolidation*, summarizing and promoting episodic traces into semantic memory and the ACE playbook, pruning stale episodes [37, 36]; (3) *gated distillation*, evaluating queued EATS candidates (with their SGC curricula) against the DP holdout and the online-FDR budget; (4) *adapter consolidation*, merging and pruning accepted ROSE adapters so the set stays bounded; and (5) an optional *federated round* (§5.1.6). STC is where the system’s most expensive, most permanent changes are made—deliberately off the interactive path, under the full gate, where a mistake is cheap to detect and reverse before morning (Figure 4.4).

5.2.4 Reversible Orthogonal Self-Edits (ROSE)

The problem with weight self-edits. SEAL shows a model can generate its own finetuning data and update its weights for persistent gains [25], but naive full-finetuning self-edits can

Mechanism	Runs	Dominant cost	Bounded by
VGSS	online, pivotal steps	N verifier calls + N suffix decodes	VoI budget; cache sharing
VoI	online, every step	one value estimate per action-type	cheap (ternary verifier)
EATS	on escalation	teacher call + candidate synthesis	escalation rate q_t
SGC	on escalation	curriculum generation	fixed small fan-out
STC	idle windows	batched distillation + merges	off interactive path
ROSE	on commit	low-rank adapter train	rank $\times$ #edits; merged
FEATS	federated rounds	DP noise + re-gating	rounds; privacy budget

Table 5.1: When each mechanism runs, its dominant cost, and what bounds it. The interactive path pays only for VoI, occasional VGSS, and rare teacher calls; the expensive, permanent work is deferred to STC.

catastrophically forget, are hard to reverse, and accumulate into an opaque, non-stationary model. ROSE makes weight-level self-improvement safe enough to ship.

Mechanism. Each accepted self-edit is realized not as a full-weight update but as a low-rank adapter *trained in a subspace orthogonal to those of prior edits*, in the O-LoRA/OPLoRA manner, so new edits minimally interfere with old ones and forgetting is bounded by the principal angle between gradient subspaces [39, 40, 41]. This yields three properties the gate relies on: *reversibility* (an edit is a separable adapter; rolling back a regression is dropping a tensor, not reconstructing a base model—Prop. 6.7); *non-forgetting* (orthogonality bounds interference, so monotonicity certified on the holdout is not silently undone elsewhere); and *boundedness* (adapters are periodically merged and pruned during STC, keeping the active set small and inference cheap [42]). ROSE thus turns the riskiest tier of self-improvement into small, composable, reversible, governance-checkable artifacts—weight edits with the operational ergonomics of skills.

5.3 Interactions and Complexity of the Seven Mechanisms

The mechanisms are not independent features; they compose, and several only make sense together.

Couplings. VoI decides *whether* a step deserves VGSS, and VGSS consumes the verifier VoI also uses to price compute, so both inherit the verifier’s accuracy. EATS produces the candidates STC gates and SGC generalizes, and ROSE is the substrate in which an accepted weight-candidate is realized so the gate’s reversibility is literally a tensor drop. FEATS reuses the entire EATS gate on incoming abstractions, which is what gives it Byzantine robustness for free. The verifier is the hub—it routes compute, prunes search, and gates learning—which is why its co-evolution (§4.3.3) is structural necessity, not polish.

What fails if a mechanism is removed. Without grammar-scoped emission, reliability collapses over long episodes; without VoI, compute is wasted and VGSS unaffordable; without the DP gate, EATS overfits the probe and degrades; without ROSE, weight edits forget and cannot be cleanly reversed; without STC, the expensive learning has nowhere off-path to run; without FEATS, the system works but forgoes the network effect. This ablation logic is RQ7 (Chapter 7.2).

The critical-path budget. A user-facing step pays for free-text cognition (sped up by self-speculation), a VoI value estimate, and—only if VoI authorizes—a bounded VGSS expansion

or an escalation. Everything expensive and permanent (gating, RL, weight edits, merges, consolidation, federation) is deferred to STC, off the interactive path. This separation is the single most important systems decision in CRUCIBLE: it is what lets a heavy self-improving stack feel light in use.

Chapter 6

Theory

We prove five things and are honest about a sixth we cannot. The upside: cost decays geometrically (Section 6.1) and federation accelerates it (Section 6.3). The safety: the self-improvement loop does not degrade competence even under adaptive, continual operation, and realized harm is bounded and reversible (Section 6.2). The efficiency: greedy VoI is near-optimal and VGSS dominates its special cases (Section 6.4). The honesty: a hard ceiling on what the teacher loop can buy, and explicit assumptions (Sections 6.3–6.4).

6.1 Geometric Cost Decay

Assumption 6.1 (Absorption). There exist $\gamma \in (0, 1]$ and $\delta_0 \geq 0$ with $q_{t+1} = (1 - \gamma)q_t + \delta_0$, where γ is the fraction of escalating mass removed next epoch by committed distillations (the generalization rate of accepted edits) and δ_0 is the arrival rate of genuinely novel hard cases (drift and irreducible difficulty).

Proposition 6.2 (Cost decay). *Under Assumption 6.1, $q_t = (1 - \gamma)^t(q_0 - \frac{\delta_0}{\gamma}) + \frac{\delta_0}{\gamma} \rightarrow q_\infty = \frac{\delta_0}{\gamma}$, and per-step cost $\mathbb{E}[C_t] = (1 - q_t)C_{\text{loc}} + q_tC_{\text{fr}}$ decays geometrically to $C_{\text{loc}} + \frac{\delta_0}{\gamma}(C_{\text{fr}} - C_{\text{loc}})$.*

Proof. The recurrence is affine with fixed point $q^* = \delta_0/\gamma$; with $e_t = q_t - q^*$, $e_{t+1} = (1 - \gamma)e_t$ gives $e_t = (1 - \gamma)^t e_0$ and the closed form. $\mathbb{E}[C_t]$ is affine in q_t and $0 \leq 1 - \gamma < 1$, so cost converges geometrically to the floor. $\square$

Remark 6.3. The floor δ_0/γ is set by novelty over absorption, both measurable (Chapter 7.2). The proposition also predicts the failure regime—highly non-stationary $\mathcal{D}_u$ ($\delta_0 \rightarrow 1$)—a testable boundary, not a hidden caveat.

6.2 The Gate: Adaptive Non-Degradation

6.2.1 Diagnosis of the Naive Gate

A natural gate commits an edit iff it clears a fixed margin on a held-out probe H . Under continual operation EATS generates *adaptively chosen* edits tested against the *same finite* H —precisely the setting of adaptive data analysis, in which holdout reuse loses validity and selection eventually overfits the probe’s noise, producing the very “self-improvement collapse” a naive analysis claims to preclude. A simple per-test concentration bound is internally valid but rests on an assumption (a clean, non-adaptive probe) that reuse silently violates. The fix must *restore* that assumption.

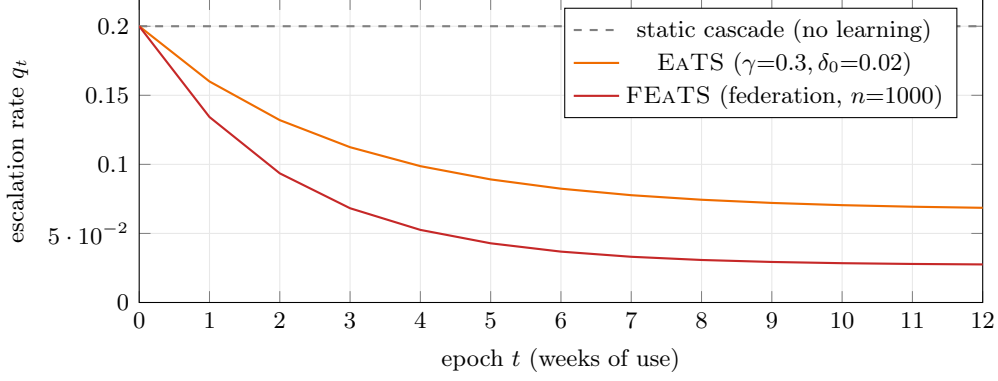


Figure 6.1: Geometric decay of the escalation rate (Prop. 6.2). A static cascade pays a fixed tax forever; EATS pays a decaying one toward floor δ_0/γ ; FEATS lowers the floor by sharing solved hard cases (Thm. 6.9).

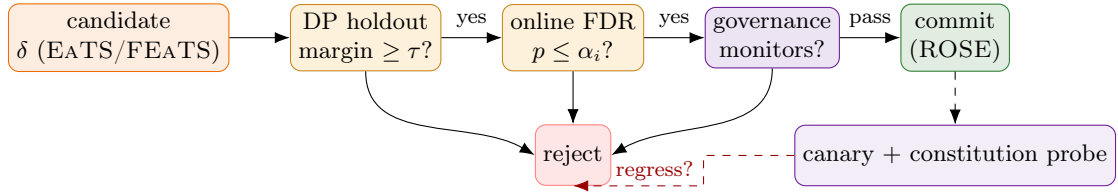


Figure 6.2: The composite gate. Per-commit validity comes from the DP holdout (Thm. 6.5); long-run acceptance is throttled by online FDR (Prop. 6.8); behavioral safety by governance; and realized harm is bounded by the reversible canary (Prop. 6.7).

6.2.2 The Composite Gate

We access H only through a differentially private mechanism, throttle acceptance with online FDR, and make every commit reversible (Figure 6.2).

Assumption 6.4 (DP-accurate holdout). The probe H ($|H| = m$, i.i.d. from $\mathcal{D}_u$, disjoint from all training and escalation data) is queried only via an $(\epsilon_{\text{dp}}, \delta_{\text{dp}})$ -differentially private, (α, β) -accurate mechanism $\mathcal{M}$ in the Thresholdout sense [53, 54].

Theorem 6.5 (Adaptive non-degradation). *Run the gate “commit δ iff $\mathcal{M}$ certifies $\hat{V}(\theta \oplus \delta; H) \geq \hat{V}(\theta; H) + \tau$ ” under Assumption 6.4 with $\tau > 2\alpha$. Then with probability $\geq 1 - \beta$, simultaneously over all K adaptively chosen candidate edits, every committed edit satisfies $V(\theta \oplus \delta) > V(\theta)$; the committed competence sequence is non-decreasing.*

Proof. By the transfer theorem of adaptive data analysis, an $(\epsilon_{\text{dp}}, \delta_{\text{dp}})$ -DP, (α, β) -accurate mechanism generalizes: with probability $\geq 1 - \beta$ its answers lie within α of true competence on all K adaptive queries at once [53, 54]. Condition on that event. A commit requires a certified margin τ , so $V(\theta \oplus \delta) \geq \hat{V}(\theta \oplus \delta; H) - \alpha \geq \hat{V}(\theta; H) + \tau - \alpha \geq V(\theta) + \tau - 2\alpha > V(\theta)$. Rejected candidates leave θ unchanged. $\square$

Corollary 6.6 (Holdout capacity). *Under Thresholdout, a probe of size m validly gates $\Theta(m^2)$ adaptively chosen edits before budget exhaustion, a quadratic improvement over $\Theta(m)$ naive [53]; beyond budget, H is refreshed (a step STC schedules).*

Proposition 6.7 (Bounded realized degradation). *With a canary C disjoint from H and never used for selection, and a checkpoint before each weight commit, if C -competence falls by more than κ within w commits, drop the offending ROSE adapters to the last C -passing checkpoint. Then realized competence never sits more than $\kappa + \text{Var}_C$ below the last certified level, regardless of any failure of the DP or FDR layers.*

Proof. Between audits the only committed changes are reversible adapters; restoration is exact (drop the tensors), and the checkpoint’s true competence is within Var_C of its C -estimate. The argument uses only reversibility and the canary. $\square$

Proposition 6.8 (FDR control is necessary but not sufficient). *Online FDR at level η bounds only the expected proportion of accepted null edits; it permits accepted harmful edits and gives no per-commit bound. Hence FDR alone cannot yield Theorem 6.5; the DP certificate and reversibility are both required. FDR remains valuable: it throttles marginal acceptances as evidence accumulates, which the per-commit test does not. (Vanilla alpha-investing controls only mFDR, and SAFFRON/LORD control FDR only under local positive dependence and conditional super-uniformity [55, 56]— conditions adaptivity violates unless valid p -values are first secured by the DP mechanism.)*

6.3 Federated Acceleration and the Weak-to-Strong Ceiling

Theorem 6.9 (Federated cost acceleration). *Consider n instances with i.i.d. novel-case streams of rate δ_0 each, sharing gate-passed abstractions under FEATS. If a fraction $\sigma \in [0, 1]$ of one instance’s novel cases are transferable once solved by any instance, then each instance’s effective novelty rate falls to $\delta_0^{\text{fed}} = \delta_0(1 - \sigma) + \delta_0\sigma/n$, and by Proposition 6.2 its cost floor falls accordingly. Every abstraction a recipient admits passes that recipient’s local gate, so Theorem 6.5’s non-degradation holds for federated candidates exactly as for local ones; and total privacy loss over r rounds is bounded by $(r\epsilon_{\text{dp}}, r\delta_{\text{dp}})$ by composition (tighter under advanced composition) [50, 53].*

Proof. A transferable novel case is solved by the frontier once across the federation; with n symmetric instances the expected per-instance share of the first solve is $1/n$, while the non-transferable fraction $1 - \sigma$ stays local, giving δ_0^{fed} . Substituting into the fixed point of Proposition 6.2 gives the floor. Non-degradation: each recipient applies its own DP-gate to every incoming abstraction, whose hypotheses concern only the gate and probe, not provenance. Privacy: each round releases through a DP mechanism; sequential composition over r rounds gives the bound. $\square$

Proposition 6.10 (Interior optimum of the DP budget). *If federated cost-saving is concave increasing in shared-abstraction fidelity, fidelity is concave increasing in ϵ_{dp} , and the privacy penalty is convex increasing in ϵ_{dp} , then the net objective is concave in ϵ_{dp} and attains its maximum at an interior $\epsilon_{\text{dp}}^* \in (0, \infty)$.*

Proof. Concave-increasing composed with concave-increasing is concave-increasing; minus a convex-increasing penalty is concave; with zero benefit as $\epsilon_{\text{dp}} \rightarrow 0$ and unbounded penalty as $\epsilon_{\text{dp}} \rightarrow \infty$, the maximizer is interior by the first-order condition. $\square$

Proposition 6.11 (Weak-to-strong ceiling). *Let $V_{\text{cov}}^{\text{fr}}$ be the teacher’s competence on covered cases and $\eta_{\text{w2s}} \in [0, 1]$ the weak-to-strong elicitation efficiency [57, 58]. The local model’s asymptotic competence on covered cases is at most $\eta_{\text{w2s}}V_{\text{cov}}^{\text{fr}}$.*

Remark 6.12. This is the honest limit: EATS/FEATS cannot exceed what the teacher elicits on covered cases. Equation (3.1) can still hold against a frozen frontier via specialization (all capacity on $\mathcal{D}_u$), coverage (most of $\mathcal{D}_u$ becomes covered, faster under FEATS), and system effects (Φ adds constrained emission, VGSS, VOI-allocated compute, verification, governed skills, consolidated memory). It is consistent with θ_t being weaker than θ^{fr} in general.

6.4 Efficiency, Assumptions, and Break-Even

Proposition 6.13 (VGSS dominates its special cases). *For a fixed node budget N and an order-consistent verifier, the action VGSS selects has verifier value at least that of best-of- N (a depth-1 tree): for equal verifier budget, search never underperforms sampling in verifier value.*

Proof. Best-of- N is the feasible VGSS policy expanding all N nodes at depth 1; VGSS’s selection maximizes estimated value over feasible allocations subject to the same budget, so its attained value is at least that of any fixed allocation, including depth-1. Order-consistency makes the $\arg \max$ respect true ordering on distinguishable actions. $\square$

Proposition 6.14 (Sub-linear VOI regret under noisy estimates). *Modeling each step’s choice as a budgeted bandit with confidence radii shrinking as $O(\sqrt{\log(1/\delta)/n_u})$, if VOI selects by an upper-confidence value-per-cost index, its cumulative shortfall against the known-value greedy over H steps is $\tilde{O}(\sqrt{H})$ with high probability.*

Proof sketch. The optimistic index yields a budgeted-bandit instance with bounded rewards and modular costs; UCB analysis sums per-step regret to $\tilde{O}(\sqrt{H})$, and cost-benefit greedy keeps the $(1 - 1/e)$ approximation against realized values [3]. Optimism prevents linear regret from a bad early estimate. $\square$

Corollary 6.15 (Probe size for an adaptivity horizon). *To gate K adaptively chosen edits at accuracy α , failure β , via Thresholdout, a probe of size $m = \tilde{\Theta}(\sqrt{K}/\alpha^2)$ suffices; equivalently size m supports $\Theta(\alpha^4 m^2)$ adaptive commits before refresh [53, 54].*

Proof sketch. Transfer-theorem generalization error scales as $\tilde{O}(\sqrt{K}/m)$; set $\leq \alpha$ and solve; the margin $\tau > 2\alpha$ then gives non-degradation at that horizon. $\square$

6.4.1 Standing Assumptions and What Would Falsify Them

(A1) *stationarity*: $\mathcal{D}_u$ drifts slowly enough that $\delta_0 < \gamma$ (else the floor approaches the frontier’s; falsified by measuring γ, δ_0). (A2) *clean probe and sound DP mechanism* (else Thm. 6.5 voids; falsified by canary regressions exceeding the bound). (A3) *accurate, hard-to-game verifier* (else VOI mis-allocates and the gate mis-certifies; falsified by verifier-accuracy and reward-hacking metrics). (A4) *teacher covered-competence exceeds the student’s* (Prop. 6.11). Each maps to a measured quantity in Chapter 7.2; none is assumed without a test that could show it false.

Corollary 6.16 (Break-even epoch). *Let a static cascade pay constant $(1 - q_0)C_{\text{loc}} + q_0C_{\text{fr}}$ and EATS add per-epoch overhead ω (the STC cost). Then EATS has lower cumulative cost after the smallest T with $\sum_{t < T} (q_0 - q_t)(C_{\text{fr}} - C_{\text{loc}}) > T\omega$, which exists whenever $\gamma > 0$ and $q_0 > \delta_0/\gamma$; under FEATS the same T is smaller because $\delta_0^{\text{fed}} \leq \delta_0$.*

Proof. Per-epoch saving is $(q_0 - q_t)(C_{\text{fr}} - C_{\text{loc}}) - \omega$, increasing toward $q_0 - \delta_0/\gamma > 0$ by Prop. 6.2, so partial sums cross zero at finite T ; replacing δ_0 by δ_0^{fed} raises each summand. $\square$

Chapter 7

Engineering, Evaluation, and Product

7.1 Implementation

CRUCIBLE is designed to be built, not only described; every component maps to an existing open artifact it integrates and extends.

Engine. SGLang is the engine of record: RadixAttention prefix sharing, co-located structured generation via XGrammar-2, and EAGLE-3 self-speculation [11, 5, 8]. CRUCIBLE adds a trajectory-grammar compiler and the two-phase decoding policy (§4.2.1); a unified-speculation scheduler with an Atomix settlement runtime (§4.2.3); a VGSS controller over the cache tree (§5.2.1); the VOI budget controller (§5.2.2); and an offline STC worker.

Training path. CG-TTRL, ROSE adapter training, and verifier co-evolution run on a veRL/HybridFlow path that hands off between SGLang inference and PPO/GRPO with minimal data movement [12]; tool-interleaved rollouts follow the Search-R1 pattern [13].

Swarm and hardware. The target policy holds the GPU; the draft head, verifier, and process-reward models can run as 1.58-bit BitNet models on CPU [34, 35], so the stack fits on one consumer machine. Pooling across machines is confined to the user’s trusted domain (§3.3.2).

Storage and scheduling. Working/episodic/semantic memory, ACE playbook, PANDO skills, ROSE adapters, probe H , and canary C are local stores (SQLite plus a vector index). STC is triggered on idle/charging; nothing leaves the device except, optionally, DP-gated federated abstractions. The orchestrator/engine contract is specified in Appendix A.2.

Deployment profiles. On a 24–48 GB workstation GPU the policy runs at 7–14 B with full VGSS depth and frequent STC; on an 8–16 GB laptop, a 4-bit 7 B with shallow, VOI-gated VGSS and charging-time STC; CPU/NPU-only runs a ternary policy with best-of-few search. Across profiles, what changes is search depth and self-edit throughput—never the correctness and safety guarantees, which rest on grammar scoping, the gate, and the monitors, not on FLOPs.

7.2 Evaluation Protocol

We pre-register research questions, baselines, suites, metrics, and analysis, and separate *component* claims from the *longitudinal* thesis claim.

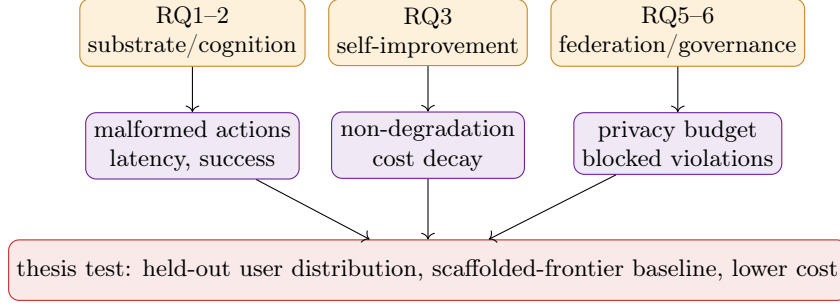


Figure 7.1: Evaluation claim matrix. Component RQs establish local mechanisms; the thesis claim is only accepted on a held-out user distribution against the identical harness wrapped around the frontier baseline.

Research questions. **RQ1 (Substrate):** does grammar-scoped emission drive malformed-action rate to 0 and cut latency $\geq 2\times$ at fixed size *without* the reasoning loss of full constraint? **RQ2 (Cognition):** do VGSS, VoI compute, and the co-evolved verifier raise success toward the frontier, and how does success scale with budget B ; what is verifier accuracy and does co-evolution keep reward-hacking near zero? **RQ3 (EATS/STC dynamics):** does the escalation rate decay geometrically (Prop. 6.2) with measurable γ, δ_0 , while committed competence stays non-decreasing (Thm. 6.5) and canary regressions stay within the bound (Prop. 6.7)? **RQ4 (the thesis):** after t^* epochs, does $\Phi(\theta_{t^*})$ exceed a frozen frontier on held-out $\mathcal{D}_u$ at lower cost (3.1)—and does it *fail* on broad/non-stationary $\mathcal{D}_u$, as predicted? **RQ5 (Federation):** does FEATS lower δ_0^{fed} and accelerate decay (Thm. 6.9) without degrading any participant or breaching the privacy budget? **RQ6 (Governance):** do monitors block specified violations before settlement, at what overhead? **RQ7 (Ablations):** per-mechanism marginal contribution.

Baselines. API-bounded local agents (OpenHands+Ollama; opencode); vanilla SGLang function-calling agent; static cascade and learned RouteLLM router [46]; the bare frozen frontier zero-shot; and—the *decisive fair baseline*—the *frontier model wrapped in the identical CRUCIBLE harness* $\Phi(\theta^{\text{fr}})$ (§7.6.2); CRUCIBLE with each layer ablated. We add **RQ8 (injection survival):** under adversarial tool outputs whose payload is a plausible-but-wrong “convention,” does the poisoning defense (§7.6.3) prevent a committed backdoor? Primary public success is measured on contamination-resistant suites (SWE-bench Pro / SWE-rebench) rather than the now-contaminated SWE-bench Verified [72, 73], with the user’s private repository carrying the longitudinal claim.

Metrics. Success; malformed-action rate; tokens, wall-clock, cost; escalation rate q_t with fitted $(\gamma, \delta_0, \delta_0^{\text{fed}})$; committed-competence trajectory on H and held-out test; accepted vs. rejected edits and rollbacks; verifier accuracy; exploit propensity and hardening effect; monitor block-rate and overhead; cumulative privacy loss.

Methodology. ≥ 3 seeds; paired bootstrap CIs on per-task deltas; Holm–Bonferroni across RQs; pre-registered effect sizes; per-epoch holdout refresh (Cor. 6.6); for RQ4, per-domain win/loss with CIs, never a single aggregate. All raw per-epoch logs released.

7.3 A Worked Example and Cost Economics

7.3.1 One Coding Task Through Six Layers

Consider “add pagination to the `/users` endpoint and update its tests,” on a laptop running a 7 B policy on GPU and a ternary verifier and draft head on CPU. (0) *Memory:* retrieve the repo’s

Suite	Role
SWE-bench Verified	Human-validated, Docker-reproducible code-agent tasks; primary RQ1/2/4 success measure.
AppWorld	Long-horizon API workflows; validates VOI/PANDO token economics and routing savings.
BFCL-v3, τ -bench	Tool-calling correctness and tool-agent dialogue; RQ1 malformed-action rate.
Reward Hacking Benchmark	Exploit propensity; environmental-hardening ablation as a headline RQ3 metric [28].
Terminal-Bench	Deep CLI/state manipulation; validates Atomix settlement under fault injection (RQ6).
Longitudinal per-domain	A fixed, narrow $\mathcal{D}_u$ replayed over epochs with strict temporal train/probe/canary/test splits; the core RQ3/4/5 instrument.

Table 7.1: Evaluation suites and roles. The longitudinal per-domain protocol, not any static leaderboard, is the instrument for the thesis claim.

ACE bullets (“this project paginates with cursor tokens”) and PANDO skills tagged **pagination**. (1) *Cognition phase*: the policy reasons in free text; self-speculation makes thinking $\sim 3\times$ faster. (2) *Emission phase*: it emits a read of **users.py**; the grammar activates for the call only, and the draft head has already speculatively issued the side-effect-free read, so contents are warm. (3) *VOI/VGSS*: choosing the scheme is pivotal, so VOI authorizes a VGSS expansion over three designs sharing one cache; the verifier picks the cursor design, consistent with the playbook. (4) *Governance*: the edit is checked against “never write outside the workspace” and settles transactionally with a compensation handler. (5) *Escalation*: an unfamiliar test fixture has low verifier confidence, so the router escalates that sub-problem to the teacher, which returns a correct fixture. (6) *Done*: tests pass; the “run tests before done” obligation is discharged. Frontier usage: one sub-problem. *That night (STC)*: the episode is consolidated into an ACE bullet (“fixtures live in **conftest.py**”), SGC expands the fixture case, the gate admits a ROSE adapter and a new skill, the canary confirms no regression, and a DP-noised skill is optionally shared. *Next week*: the same fixture pattern recurs and is handled locally—its cost has dropped from C_{fr} to C_{loc} , exactly Proposition 6.2, and under FEATS other users stopped escalating it too (Thm. 6.9).

7.3.2 Cost Economics with Worked Numbers

Representative (not measured) figures show the shape of the argument. Suppose 200 steps/day, hard fraction $q_0 = 20\%$, $C_{\text{fr}} = \$0.03$, $C_{\text{loc}} = \$0.0002$. A static cascade costs $\approx 200(0.8 \cdot 0.0002 + 0.2 \cdot 0.03) = \$1.23/\text{day}$. With $\gamma = 0.3/\text{week}$, $\delta_0 = 0.02$, Proposition 6.2 gives floor $q_\infty \approx 6.7\%$ and steady-state spend $\approx \$0.44/\text{day}$ —a $\sim 64\%$ reduction, bought once and retained. With $\sigma = 0.6$ over $n = 1000$, $\delta_0^{\text{fed}} \approx 0.008$, floor $\approx 2.7\%$, spend $\approx \$0.18/\text{day}$ —an $\sim 85\%$ reduction (Figure 6.1). The break-even epoch (Cor. 6.16) is where cumulative savings overtake the STC overhead ω —days to weeks at these rates. The point is structural, not the dollars: a static cascade pays a fixed tax forever; EATS pays a decaying one; FEATS lowers the floor.

7.4 Criticisms, Limitations, and Ethics

7.4.1 Anticipated Criticisms and Rebuttals

- O1. “A 7B cannot beat a frontier model.”** We claim only Eq. (3.1): the *system* on the *user’s* distribution after specialization, bounded by Prop. 6.11, predicted to fail on broad $\mathcal{D}_u$ (RQ4).
- O2. “Self-improvement collapses via multiple testing.”** Fixed by the DP holdout (Thm. 6.5), online FDR, and reversibility (Prop. 6.7); collapse probability is bounded and tunable.
- O3. “Alpha-investing alone fixes O2.”** No: it controls mFDR, not non-degradation, and presupposes valid p -values the DP mechanism supplies (Prop. 6.8).
- O4. “Constrained decoding destroys reasoning.”** Only when reasoning is forced into a schema; we constrain emission, not cognition (§4.2.1), and emission constraint is essential given compounding session failure [6, 7].
- O5. “Unsupervised RL reward-hacks.”** EATS is teacher-grounded; spurious rewards are model- and uninformative-reward-specific [26]; the verifier is co-evolved [17, 19]; hacking is measured and hardened [28].
- O6. “Latent reasoning is unproven.”** Excised; cited as a negative result [29].
- O7. “Speculative execution corrupts state.”** Atomix verify-then-settle with compensation; only side-effect-free actions pre-execute [9].
- O8. “Skill libraries bloat.”** PANDO lifecycle: keyword retrieval, polarity-pair merge, confidence demotion [22].
- O9. “Weight self-edits forget / are opaque.”** ROSE: orthogonal, separable, reversible, merged during STC [39, 40, 42].
- O10. “Static cascade is inefficient.”** Learned preference router with calibrated thresholds and transfer [46].
- O11. “Decentralize via Petals.”** Rejected for cross-tenant use: activation pooling leaks inputs [51, 52]; pooling is trusted-domain only, federation shares only DP abstractions (§5.1.6).
- O12. “Federation leaks data or invites poisoning.”** Only DP, gate-passed abstractions are shared; recipients re-gate every update (Thm. 6.9); privacy is composed.
- O13. “Privacy is marketing.”** Architectural: Def. 3.2(iv), on-device STC, DP-only sharing.
- O14. “The verifier is a single point of failure.”** Acknowledged (A3); mitigated by co-evolution, isomorphic checks, reversibility; measured (RQ2).
- O15. “VoI estimates are unreliable.”** Near-optimality is conditional on verifier accuracy (Prop. 6.14); falls back to uniform spend when confidence is low.
- O16. “Too complex / irreproducible.”** Strict layer dependency; each maps to an open artifact; phased build (§7.7.2).
- O17. “Benchmarks are gameable / leak.”** SWE-bench Verified + longitudinal temporal splits + explicit reward-hacking measurement.
- O18. “Governance monitors are just prompts.”** No: compiled temporal-logic specs enforced at the settlement boundary with pre-execution guarantees [43, 44].

7.4.2 Limitations and Threats to Validity

Beyond the standing assumptions (§6.4.1): *stationarity* is the binding empirical risk; *verifier quality* upper-bounds gating, routing, and VOI; *teacher dependence* means amplification inherits covered errors; *engineering surface* is large, and partial deployments (Layers 1–2) do not test the thesis; *federation* assumes enough cross-user overlap ($\sigma > 0$); and a *DP-utility tension* means strong privacy slows transfer (Prop. 6.10 locates the optimum rather than hiding the tension).

7.4.3 Ethics, Safety, and Broader Impact

Self-modification is gated, reversible, and checkpointed, with a frozen constitution probe in the canary so safety-degrading edits are rejected by the same mechanism as competence regressions. Tools run under least-privilege sandboxes with temporal-logic monitors (Chapter 4.7). Data stays on device; only DP abstractions are shared; cross-tenant activation pooling is prohibited. The loop expands competence within a fixed, sandboxed action space, not the action space itself, and all self-edits and escalations are logged immutably. By making capable agents run privately on consumer hardware while still enjoying collective improvement, CRUCIBLE reduces dependence on centralized providers without forcing a choice between privacy and progress.

7.5 Adversarial Hardening: Failure Modes and Their Resolutions

We subjected CRUCIBLE to a systems-and-theory red-team. This chapter records each failure mode it surfaced, our disposition—ACCEPT (a real defect; fix it), REFINES (right direction, corrected framing), or REFUTE (the objection overreaches; here is the counter-evidence)—and the resolution. Where we refute, we bring our own proof rather than assertion. The resolutions here are *normative*: they refine the corresponding earlier chapters. Net disposition: seven accepts, two refinements, one refuted conclusion, summarized in Table 7.2.

7.5.1 Fault Isolation: Co-Location Without a Shared Crash Domain

ACCEPT. The critique is correct that placing arbitrary agentic logic in the *same process* as the engine is unsafe: an orchestrator memory leak during a deep VGSS expansion would crash the engine and destroy the shared KV-cache. Our earlier language (“shared address space”) overstated the coupling. The resolution is that “inference-native” (Def. 3.2) means *shared state access*, not a shared fault domain. Engine and orchestrator run as separate processes; the cache tree, logits, and adapters are exposed through *zero-copy shared memory and IPC* (handles, not copies), so the orchestrator reads and forks the cache at memory speed yet cannot take the engine down with it (Figure 7.2). The engine keeps doing the stateless tensor work it is optimized for; the stateful, transactional application logic lives in its own process with its own resource limits and a supervisor that restarts it without losing engine state. This preserves every capability of Def. 3.2 while removing the single-crash-domain hazard.

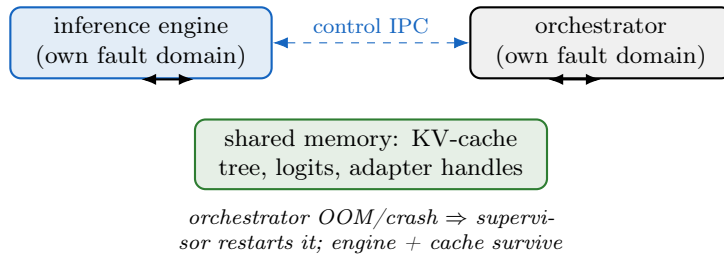


Figure 7.2: Fault-isolated co-location: shared-memory state access (zero-copy) with separate crash domains, replacing the single-process reading of Def. 3.2.

Failure mode	Disposition	Resolution
Memory co-location crash	ACCEPT	zero-copy IPC; engine and orchestrator in separate fault domains (§7.5.1)
Structure snowballing	ACCEPT	failure-recovery diagnostic mode; “In-Writing” two-phase confirmed (§7.5.2)
Atomix compensation fails	ACCEPT	reversibility classes + micro-VM epoch snapshots + learned reversibility profiling (§7.5.3)
Chain-of-Draft over-use	REFINE	already VOI-modulated; sharpened and bounded to easy steps (§7.5.4)
Sleep-time hallucinated drift	ACCEPT	bipartite reconstruction gate + verified-only consolidation (§7.5.5)
Holdout exhaustion voids cost-decay	REFUTE	execution is a <i>free</i> oracle; holdout refreshed without paid teacher (§7.5.6)
ROSE rank saturation	ACCEPT	SLAO continual merging; rank pressure scales with novelty, not task count (§7.5.7)
Federated DP noise amplification	ACCEPT	FedASK two-stage sketching + subspace-aware DP + symbolic-primary channel (§7.5.8)
VGSS variance / reward hacking	ACCEPT	RTV/PDR summary-level search + execution anchor + search/gate separation (§7.5.9)
Open-source cedes the moat	REFINE	tiered licensing; the moat is downstream, not the license (§7.5.10)

Table 7.2: Red-team scorecard. Each row links to its resolution.

7.5.2 Structure Snowballing: A Failure-Recovery Diagnostic Mode

ACCEPT (and largely anticipated). Forcing reasoning into a grammar can crystallize a logical error into a perfectly formatted, wrong action—“structure snowballing” [65]. CRUCIBLE’s two-phase policy already keeps cognition unconstrained, and the independent “In-Writing” result confirms the design: reason freely until a trigger token, then constrain only the emission, for up to +27% accuracy over constrained baselines [66]. We add the missing piece the critique identified—an explicit *failure-recovery diagnostic mode*: on a tool-execution failure or a verifier rejection, the runtime *drops all grammar constraints* for the following reflection, lets the model diagnose in free natural language, and *penalizes superficial syntactic fixes*; the grammar mask is re-engaged for the next emission only after the process verifier confirms the semantic error was actually restructured. Reflection is cognition, and CRUCIBLE never constrains cognition—now including the cognition that happens *after* a failure.

7.5.3 Transactional Reality: Reversibility Classes and Micro-VM Snapshots

ACCEPT. The critique is right that Atomix’s clean reversible/irreversible split is idealized: compensation handlers are themselves API calls that hit rate limits and timeouts, and “read-only” calls can trigger receipts and consume quotas. Three hardenings: (i) *reversibility classes*—every tool is typed as *idempotent*, *compensable*, *snapshot-reversible*, or *irreversible*; only the first three are ever speculated, and irreversible actions require confirmation and never run on a speculative branch. (ii) *Micro-VM epoch snapshots*—for local state (files, terminal), each Atomix epoch is bound to a Firecracker-class snapshot; an abort restores the exact pre-epoch memory and disk state in milliseconds rather than running a reverse script [70]. (iii) *Learned reversibility profiling*—the co-evolved verifier tracks the empirical success rate of each endpoint’s compensation

across the federation; when a handler proves unreliable, VOI downgrades that tool’s reversibility confidence and forbids it in deep speculative branches. Quota- and receipt-bearing reads are tracked as *compensable-with-cost* rather than free. Reversibility becomes a measured property, not an assumption.

7.5.4 Chain-of-Draft, Bounded

REFINE. The objection—that five-word drafts dilute the deep state-tracking needed for multi-hop coding—is correct in general but already addressed in design: CRUCIBLE never applies Chain-of-Draft uniformly. The VOI controller sets the reasoning budget per step (§4.4.3); brevity is enforced only on low-perplexity, single-file steps, and is *suspended*—restoring full free-text reasoning or a VGSS expansion—whenever the process-reward score drops or the step touches multiple files or cross-scope dependencies. We sharpen the rule here and tie it to the agentic-coding evidence that long-horizon tasks need trajectory-level, not token-minimal, representations [67]. Compression is a dial VOI turns down on hard steps, never a blanket constraint.

7.5.5 Sleep-Time Drift: A Bipartite Reconstruction Gate

ACCEPT. The risk is real: offline consolidation can distill a daytime misunderstanding into a confident “axiom” that poisons future reasoning, and the constitution probe tests safety, not the factual fidelity of user-local memories. Two hardenings: (i) a *bipartite reconstruction gate*—after STC produces a condensed summary, a second, differently-conditioned process must reconstruct the original episodic trace *from the summary alone*; if semantic divergence from the raw trace exceeds a threshold, the summary is rejected as lossy/hallucinated and never enters semantic memory. (ii) *Verified-only consolidation*—only episodes whose outcomes were execution-verified or teacher-confirmed are eligible for consolidation; uncertain or failed traces are quarantined. Consolidations are, moreover, gated by the same DP holdout and canary as any other edit (Chapter 6.2): a memory is just another edit type, and edits must prove they help.

7.5.6 Holdout Economics: Execution is a Free Oracle

REFUTE (the conclusion), ACCEPT (the finiteness). The critique’s strongest claim is that the differentially private holdout has a finite adaptivity budget, that refreshing it needs fresh ground-truth labels, and that a local model can only obtain those by paying the frontier teacher—so the geometric cost-decay (Prop. 6.2) is “voided” and the user is “bankrupted.” The finiteness is real (Cor. 6.6); the conclusion is false, and here is the proof.

Key idea. In CRUCIBLE’s target domains the ground-truth oracle is the *environment*, not the teacher. Reinforcement learning with verifiable rewards (RLVR) shows that code execution, unit tests, compilers, and formal checks are deterministic, automatically verifiable reward signals that *replace* human or teacher judgment [68]. So the holdout is refreshed from execution-verified task outcomes that occur *for free* as the user works—no extra frontier call required.

Concretely, the probe H and canary C are refilled from three free or cheap sources, in priority order: (1) *execution-verified outcomes*—every task the user runs that has a test/compile/run result is a fresh, i.i.d. labeled sample from $\mathcal{D}_u$ at zero marginal cost [68]; (2) *opportunistic escalation labels*—escalations happen anyway as part of normal operation, and their verified results are a free byproduct; (3) *active human curation*—when budget runs low, the system selects a small, maximally diverse (high-information-gain) set of tasks and asks the developer to confirm them, turning refresh into an explicit, occasional, high-value step rather than a hidden cost sink.

The frontier teacher is the *last* resort, not the refresh mechanism, so Proposition 6.2 stands. We concede the honest boundary: in domains with *no* cheap verification oracle (no tests, no execution, purely subjective tasks), the budget does bind and refresh becomes expensive—there, CRUCIBLE’s self-improvement is correspondingly weaker, which assumption A2/A3 (§6.4.1) already flagged. For the agentic-coding and tool-use distributions CRUCIBLE targets, the oracle is free.

7.5.7 ROSE Without Rank Collapse: SLAO and Novelty-Bounded Pressure

ACCEPT. Orthogonal adapters consume finite subspace dimensions, and naive merging of orthogonal adapters destroys the geometry that protected them. Resolution: CRUCIBLE adopts Single-LoRA continual merging (SLAO)—each new task’s adapter is initialized from orthogonal bases extracted from the *already-merged* adapter, and merging exploits the LoRA A/B asymmetry (the B update is empirically more orthogonal to its initialization), so capability is consolidated into one bounded adapter without naive averaging [42]. Two further points bound the rank pressure. First, it scales with *novelty*, not task count: related skills share subspaces, so only genuinely new capabilities claim new dimensions, and the novel-case rate δ_0 (Assumption 6.1) is small in a stationary domain—the same quantity that makes cost decay. Second, STC prunes low-value adapters by the same gate metric, so the active rank budget is a managed resource, not a monotonically growing one. Rank saturation is thus a measurable, governed pressure, not an inevitability.

7.5.8 Federation Without Utility Collapse: FedASK and a Symbolic-First Channel

ACCEPT. The critique correctly identifies the privacy–utility dilemma: adding DP noise to both LoRA factors and then *averaging* amplifies noise multiplicatively and can null the global model’s utility, so federated updates would fail every recipient’s gate and the network effect would die. Two hardenings. (i) Replace naive averaging with *FedASK*-style two-stage sketching: a randomized-SVD pipeline aggregates sketched, privacy-preserving updates and reconstructs the global factors on the server, achieving *exact aggregation with a proven DP guarantee*—empirically +11.5% MMLU and +46% GSM8K under strong DP versus baselines [69]. (ii) Make the *symbolic* channel primary: ACE playbook bullets and PANDO skills are redactable text, not matrices, so they carry no matrix-multiplication noise amplification at all; the network effect can run mainly on these low-noise symbolic abstractions, with numeric adapters shared secondarily via FedASK. We add *subspace-aware DP*: noise is scaled inversely to how common a task subspace is, so widely shared syntax skills keep their utility while idiosyncratic, sensitive local logic gets maximum protection. Utility and privacy are co-optimized rather than traded one-for-one.

7.5.9 VGSS: Summary-Level Search, an Execution Anchor, and the Search/Gate Firewall

ACCEPT, with an important defense the critique missed. Action-level MCTS is the wrong granularity for long-horizon coding, and a co-evolved neural verifier can be gamed (“synergistic collapse”). Resolutions: (i) CRUCIBLE searches at the *trajectory-summary* level using Recursive Tournament Voting and Parallel-Distill-Refine—each rollout is compressed to its salient hypotheses and failure modes, and a population is narrowed by small-group comparison rather than point-wise PRM scoring, which is exactly what lifts frontier agents on SWE-bench Verified and Terminal-Bench [67]. (ii) *Execution grounding is the anchor*: whenever a verifiable signal exists (compiler, tests, runtime), it *overrides* the neural PRM, an unhackable deterministic reference [68, 19].

Key idea. The decisive defense against reward hacking is a *firewall* between search and commitment. The co-evolved verifier *guides search*—where collusion is tolerable because it only steers a heuristic that is then checked. But the *gate* that commits learning uses *environmental* ground truth (execution-verified holdout, §7.5.6), which the policy-verifier pair cannot collude to fake. So a verifier blind spot can waste a little search compute, but it *cannot* propagate into a committed weight edit. Synergistic collapse is contained by construction.

7.5.10 Licensing: The Moat Is Downstream, Not the License

REFINE. The objection—open-sourcing the spin-outs lets cloud incumbents absorb them in a quarter—has a kernel of truth and a flawed conclusion. The kernel: a tiered license is prudent, so CRUCIBLE keeps the *primitive* libraries permissively licensed (trust and adoption are the wedge, and the breakout local-AI projects of 2026 are permissive OSS), while the *integrated server* and the *enterprise* components (federation coordinator, governance) use a time-delayed Business Source License that converts to open source after a fixed term—monetizable now, open eventually, and never a trap for individual developers. The flawed conclusion is that open primitives “cede the moat.” They do not, because the load-bearing moats are downstream and license-independent: (a) the grammar/transactional primitives are *already* open in the literature (XGrammar, ToolDec, Atomix), so CRUCIBLE cedes nothing by open-sourcing its versions; (b) the durable moats are the *integrated experience*, the *on-device specialization data* (Products 5–6), and the *private network effect* (Product 8)—none of which a competitor obtains by absorbing an MIT-licensed library, and the last of which a cloud’s trust model structurally cannot replicate. Open primitives *widen the funnel*; the moat was never the license.

7.5.11 Net Effect

The red-team made CRUCIBLE materially stronger: fault isolation removes a crash hazard, micro-VM snapshots and reversibility classes make speculation safe against real APIs, the bipartite gate stops memory drift, SLAO and FedASK rescue the evolution and federation layers from rank and noise collapse, and summary-level search with an execution anchor and the search/gate firewall defuses reward hacking. The one conclusion we reject—that the holdout dooms the economics—falls to a single fact the critique overlooked: in verifiable domains, truth is free. None of these resolutions weakens the thesis; each removes a way it could have failed.

7.6 Adversarial Hardening II: Coherence, Evidence, and the Poisoning Channel

A second, deeper red-team—run before external review—surfaced failure modes more serious than the first, clustered in five areas the first pass missed: a structural contradiction between the frozen-model Substrate and the model-changing Evolution layer; an *evidentiary* flaw in how the central claim is tested; a poisoning channel through the self-improvement loop; the fragility of orthogonality as a safety guarantee; and several theory results whose stated assumptions the system violates. We treat these in the same spirit as Chapter 7.5: concede what is real, fix it, and—where two reviewers genuinely conflict—adjudicate with our own judgment and say so. Some of these are *open problems*; we mark them as such rather than claim victory, because a self-improving system that hides its residual risks is exactly the kind a careful reviewer should reject (Table 7.3).

Failure mode	Disposition	Resolution
Self-edits invalidate cache/draft/steering	ACCEPT	batch edits to STC boundary; aLoRA base-cache; re-align/re-extract nightly (§7.6.1)
Eq. 1 baseline rigged (unscaffolded frontier)	ACCEPT	add $\Phi(\text{frontier})$ arm; reframe claim around cost/privacy/specialization (§7.6.2)
SWE-bench Verified contaminated	ACCEPT	primary $\rightarrow$ SWE-bench Pro/rebench; private-repo longitudinal (§7.6.2)
Net token savings overstated	ACCEPT	report net tokens incl. verifier/VGSS/SGC (§7.6.2)
Learning loop is a poisoning channel	ACCEPT (open)	provenance quarantine; backdoor screening; symbolic-only federation (§7.6.3)
Orthogonality $\neq$ safety preservation	ACCEPT	alignment-preserving LoRA; fresh randomized safety probe (§7.6.4)
DP-holdout i.i.d. violated by drift/correlation	ACCEPT	independent reservoir refresh; reversibility as primary net (§7.6.5)
VGSS dominance / VoI submodularity / FEaTS lifetime ε	REFINE	conditional statements; honest constants (§7.6.5)
VGSS not “near-free”; laptop OOM	ACCEPT	IO-aware tree attention; re-spec tiers; report peak memory (§7.6.6)
Wedge squeezed (MoE, price collapse); BSL fork-bait	REFINE	MoE-/size-agnostic; privacy-anchored; permissive license (§7.6.7)

Table 7.3: Second red-team scorecard.

7.6.1 Coherence Under Self-Edits: The Frozen/Changing Contradiction

ACCEPT — this is the deepest finding, and two independent reviewers reached it separately. The Substrate’s speedups (RadixAttention prefix sharing, EAGLE-3 self-speculation, DroidSpeak/KVComm cross-agent cache reuse, activation steering) all presuppose a *fixed* model, because KV-cache entries, draft-head features, and steering directions are all functions of the weights [74, 75]. The Evolution layer (ROSE) exists to *change* the weights. Stated naively, every adapter commit invalidates the cache, decays the draft head, and drifts the steering vectors—so the headline speedups and the headline self-improvement appear mutually corrosive.

Key idea. The resolution is a scheduling invariant we already half-imposed and now make load-bearing: **the model is frozen *within* a session; weights change only at STC boundaries.** ROSE commits, merges, and prunes happen exclusively during sleep-time consolidation, never mid-session. So during interactive use the adapter set is fixed and every cache/draft/steering assumption holds. Coherence is restored once per cycle, not corrupted per step.

Concretely: (i) *Cache*. Within a session the cache is valid; at each STC boundary it is versioned by active-adapter-set, and we adopt Activated-LoRA so the *base-context* KV survives adapter activation (only the adapter-induced delta is recomputed), preserving most cross-session reuse rather than a full flush [74]. (ii) *Draft head*. EAGLE-3’s head is target-specific and decays as the target drifts [75]; we therefore add draft-head re-alignment as a booked STC step (parameter- and data-efficient [75]), and we *retract the 2–6 $\times$ figure for the edited regime* unless re-alignment is run—honesty requires booking this in the overhead ω (Cor. 6.16). (iii) *Cross-tenant reuse*. DroidSpeak reuses only layers whose KV is close enough—as few as 13–15 of 32

even for mild instruction-tuning, with heavy divergence listed as future work—so under per-user ROSE divergence cross-tenant KV reuse is *unsound*; we restrict KV reuse to agents sharing the *same* adapter stack (homogeneous pools) and otherwise fall back to the symbolic message channel, dropping the cross-tenant KV claim. (iv) *Steering*. Control vectors are re-extracted against the current adapter stack during STC (cheap, contrastive), and stay ablatable. The contradiction becomes a *managed once-per-cycle coherence operation*; we add it as a first-class STC stage and as a measured cost.

7.6.2 Evidentiary Integrity: A Fair Baseline and an Uncontaminated Benchmark

ACCEPT — and this is the most important honesty fix in the treatise. The central claim (Eq. (3.1)) was to be tested against a *frozen frontier*, *zero-shot* baseline. That is not a test of “architecture closes the parameter gap”; it is a test of “a scaffolded small model beats an *unscaffolded* large one,” which is a foregone conclusion—scaffolding variance now exceeds model-to-model variance, and the same model can move tens of points on SWE-bench by changing only its harness [71]. We correct this:

Key idea. The fair test gives the *frontier model the same scaffold*. RQ4 now has three arms: $\Phi(\theta_{\text{local}})$, $\Phi(\theta^{\text{fr}})$ (the frontier model inside the identical CRUCIBLE harness, minus the on-device weight edits it cannot receive), and the bare frozen frontier. The honest, defensible claim is not “local beats frontier” in the abstract; it is: *after specialization, $\Phi(\theta_{\text{local}})$ matches or beats $\Phi(\theta^{\text{fr}})$ on the user’s own distribution at $\gg$ lower cost and with full privacy*—and we expect, and will report, that a scaffolded frontier wins on cold/general tasks. The win is specialization, cost, and privacy, not raw capability.

Two further evidentiary fixes. *Contamination*: SWE-bench Verified is now known to be contaminated—models identify the buggy file from the issue text alone far above chance, and scores collapse on decontaminated splits [72]—so we move the primary public measure to dynamic, post-cutoff suites (SWE-bench Pro / SWE-rebench) [73] and lean on the user’s *private* repository for the longitudinal claim, which is uncontaminated by construction. *Net tokens*: the “5–10 $\times$ ” figure counted reasoning compression but not the tokens *added* by the generative verifier, VGSS search, and SGC; we now report **net end-to-end tokens at matched task success** as the headline metric, and state plainly that on hard long-horizon tasks the honest factor is likely small-integer, not 5–10 $\times$ (the large factor holds for easy, verification-light steps). The Chain-of-Draft figure is, moreover, math-derived; we no longer cite it as agentic evidence (§7.5.4).

7.6.3 The Poisoning Channel: When Learning Ingests the Adversary

ACCEPT, and partly an *open problem* we will not paper over. CRUCIBLE’s defining feature—a gated self-improvement loop—is precisely the surface that 2026 attack papers target. Untrusted tool/web output enters an escalated trajectory, and EATS can distill that trajectory into a skill or a ROSE weight edit; a backdoor that keeps *clean-input outputs correct* passes a competence gate by construction, and FEaTS could then broadcast it [76, 77, 78]. “Zombie Agents” explicitly targets weight-/skill-updating agents and circumvents gating, verification, and filtering; BadSkill reports >90% attack success with intact benign accuracy [76, 78]. Crucially, *teacher-grounding does not help when the poison is co-present in the escalated context*—we concede this directly.

Mitigations (defense-in-depth, not a proof of immunity): (i) *Provenance quarantine* — taint-track untrusted spans; any candidate edit *derived from* a trajectory that ingested untrusted content is quarantined and may become a commit only via the teacher’s *independently re-derived* solution on sanitized context, never by copying the untrusted span. (ii) *Backdoor*

screening orthogonal to competence—because the gate measures clean competence (which a backdoor preserves), we add trigger-scanning, activation-clustering, and adapter weight-space anomaly detection as separate gate stages, and state honestly they are imperfect. (iii) *Federation hardening*—we withdraw the “Byzantine-robust by construction” claim; cross-trust-boundary sharing is restricted to the *symbolic, human-auditable* channel (skills/playbook), opaque numeric adapters are shared only *within* a trust domain, and consolidation requires multi-party agreement plus anomaly detection. This weakens the adapter-based network effect, so the network effect now rests primarily on auditable symbolic abstractions—a smaller but honest claim. (iv) A dedicated *injection-survival red-team* (RQ8, §7.2) measures the residual. *Honest residual*: stealthy trigger-conditioned backdoors are an unsolved problem in the field; CRUCIBLE reduces the attack surface and adds detection, but does not claim immunity, and a deployment that cannot tolerate this risk should run with self-edits disabled (Products 1–4 still function).

7.6.4 Safety Under Self-Edit: Orthogonality Is Not a Safety Guarantee

ACCEPT. ROSE claimed that orthogonal edits keep safety intact and that a *frozen* constitution probe catches regressions. Both are wrong. “The Geometry of Alignment Collapse” shows orthogonality to safety-critical directions is structurally unstable under gradient descent and gives “false reassurance”; even benign fine-tuning degrades guardrails, and merging third-party LoRA can erase base-model alignment [79]. Fixes: (i) use alignment-preserving adapters that explicitly project edits off measured safety subspaces (AlignGuard-LoRA, Safe-LoRA) [80, 81]; (ii) replace the frozen probe with a *freshly randomized* safety probe each refresh (a fixed probe is gameable and under-samples the behavior space), and re-certify safety *per merge*, not from geometry; (iii) treat any safety regression on the randomized probe as a hard rollback trigger (Prop. 6.7). *Honest residual*: a finite probe cannot cover all directions; randomization and expansion shrink, but do not eliminate, the blind spot—another reason self-edits are reversible and off by default for risk-averse deployments.

7.6.5 Theory Corrections

ACCEPT/REFINE, with honest restatements. (a) *DP-holdout i.i.d.* The transfer theorem behind Thm. 6.5 needs an i.i.d. holdout, but drift (A1) and refresh from execution-verified tasks that *also seed candidates* (§7.5.6) break both independence and stationarity. Fix: refresh H by *random reservoir sampling* from the task stream, and forbid any task that contributed to a candidate from entering H (independence by construction); demote the DP guarantee to “under stationarity,” and elevate *reversibility* (Prop. 6.7) to the *primary* safety net, which holds regardless of distributional assumptions. (b) *VGSS dominance*. Prop. 6.13 assumed an order-consistent verifier; under a noisy verifier, deep search can underperform breadth via error propagation [82]. We restate it as conditional on a *measured* order-consistency bound and default to breadth (best-of- N) when verifier calibration is low. (c) *VoI submodularity*. The $(1 - 1/e)$ guarantee needs monotone submodular value, which correlated compute-actions (and the threshold/super-modular escalation action) violate. We state submodularity as an explicit, tested assumption, handle escalation as a separate non-modular option, and report the honest non-monotone constant when it fails. (d) *FEaTS lifetime privacy*. Composition over r rounds gives $(r\varepsilon, r\delta)$ (or $O(\sqrt{r}\varepsilon)$), unbounded as the system runs forever; under a fixed *lifetime* budget the per-round $\varepsilon \rightarrow 0$, which re-erodes FedASK’s utility. Fix: a lifetime privacy accountant with time-adaptive allocation, and an explicit claim that compounding returns survive the decaying per-round ε *only if* the transferable-novelty rate also decays—which we now state as an assumption to test, not a guarantee. (e) *Holdout constants*. The $\Theta(m^2)$ capacity is asymptotic with bad constants; at a consumer-scale probe we publish a concrete operating-point table (m, α, τ , edits-per-refresh) and concede that if the gate cannot keep pace with δ_0 , the absorption rate γ is capped—bounding, honestly, how fast cost can decay.

7.6.6 Systems Realism

ACCEPT. VGSS is not “near-free”: prefix sharing saves shared prefill, not the $W \times D$ divergent suffix decodes (memory-bandwidth-bound), the tree-attention KV-IO blowup (redundant several-fold without IO-aware kernels), or the per-node *generative* GenPRM pass (itself a reasoning rollout) [83]. Fix: adopt DeFT-style IO-aware tree attention [83]; re-baseline VGSS cost as $W \times D$ generative-verifier passes plus divergent decodes; and have VOI price *that* real cost, not “one value estimate per action-type.” Laptop deployment: a 4-bit 7B with a 32K KV-cache already approaches an 8 GB budget before draft/verifier/VGSS load, so the 8–16 GB tier is re-spec’d honestly—KV-cache quantization, small context, *no* resident VGSS tree (best-of-few only), draft time-sliced—and **peak resident memory** becomes a reported eval metric. The full stack is a workstation-tier (24–48 GB) configuration; we no longer imply the laptop runs it whole.

7.6.7 Market Repositioning and a Licensing Adjudication

REFINE. Two market facts erode the original wedge: cloud token prices are collapsing (making a token-dollar pitch a shrinking absolute number), and local *mixture-of-experts* models now deliver 26–35B-class quality at ~3B active compute on consumer GPUs [86]. Corrections: (i) the wedge is re-anchored on *privacy and on-device specialization data*, not token dollars, and led with those; (ii) the architecture is stated as *model-size-agnostic and MoE-native*—“any local model that fits,” explicitly including sparse 20–40B—rather than fixated on dense 7–9B; (iii) we concede Products 0–1 are low-moat loss-leaders and compress the timeline to the first *defensible* rung (Memory/Evolve: switching cost + specialization data).

Licensing adjudication (the two reviewers conflict). One reviewer urged a delayed Business Source License to stop incumbents absorbing the spin-outs; another showed BSL is fork-bait that erodes exactly the trust this product sells (OpenTofu forked HashiCorp; Valkey reached majority adoption within a year of Redis’s relicense). We side with *permissive licensing throughout the user-facing stack*, and monetize the *managed service* (hosted federation, governance, escalation routing, support, compliance), not a restrictive license. The reasoning is our own: for a privacy- and trust-positioned product, a non-OSI license is a self-inflicted wound, and—per Chapter 7.5—the durable moat was never the license but the integrated service, the specialization data, and the network effect. Open everything a developer runs locally; charge for what we operate.

7.6.8 Net Effect and Honest Residuals

This pass changed substantive claims, not just wording: a fair (scaffolded) baseline and an uncontaminated benchmark, a net-token metric, a coherence invariant that reconciles the Substrate with self-edits, alignment-preserving edits with a randomized safety probe, and a federation that shares auditable symbolic abstractions rather than opaque adapters across trust boundaries. Three *honest residuals* remain, and we state them as the research frontier rather than hide them: (1) stealthy trigger-conditioned backdoors through the learning loop are not fully solved—self-edits are reversible and optional for this reason; (2) safety preservation under continual self-edit is bounded by a finite probe; (3) the federation’s value is bounded by the privacy–utility frontier. The thesis survives in a narrower, more defensible form: *a local model, in a fair harness, can match a scaffolded frontier on its user’s distribution at far lower cost and with full privacy, and improve toward surpassing it through gated, reversible specialization*—which is a claim we can actually defend.

7.7 Product, Positioning, and Conclusion

7.7.1 A New Category, Not a Better Wrapper

The visible wave of local agent software are, architecturally, *clients*: they speak to a model server over a text protocol, and their innovations live *above* the model. CRUCIBLE is not a better client; it is a *runtime* that erases the client/server split for the components that matter, relocating control to where the information is—the move operating systems made over batch monitors. The resulting capabilities (grammar-scoped emission, unified speculation, verifier-guided search, value-of-information compute, reversible self-edits, sleep-time consolidation, private federation) cannot be added to an API-bounded tool, because they require state the API discards. The wager is that the first system to put judgment below the wall, improve it safely, and let it compound privately and collectively will define the category—and that the timing is right because capable small models, the requisite research, and edge demand have all arrived.

7.7.2 Product Roadmap

P0 — Substrate MVP: SGLang + grammar-scoped emission + shared-cache loop; a local coding agent with 0% malformed actions (RQ1), usable on its own. **P1 — Cognition:** co-evolved verifier, VoI, unified speculation with Atomix, VGSS (RQ2/6). **P2 — Memory:** three-tier memory + STC. **P3 — Evolution (safe tiers):** ACE + PANDO via EATS + SGC. **P4 — Evolution (weights) + routing:** learned router; gated CG-TTRL, then gated ROSE with the full DP+FDR+rollback gate (RQ3/4). **P5 — Federation:** FEATS (RQ5)—the network-effect moat.

7.7.3 Related Systems

System	Below wall (logits/cache)	Constrains emission only	Self-improves (weights)	On-device private	Federated (DP)
OpenHands / open-code	no	via prompt	no	partial	no
Odysseus-style WS	no	no	no	yes	no
Cloud agents	n/a	varies	no (frozen)	no	no
CRUCIBLE	yes	yes	yes (gated)	yes	yes

Table 7.4: CRUCIBLE versus representative systems. The first column is the dividing line.

7.7.4 Conclusion

The frontier gap for local agents is mostly a gap of *architecture*, not *parameters*: prevailing tools orchestrate above an API wall that hides exactly the control surfaces small models need. CRUCIBLE pushes the loop beneath the wall and assembles six layers on the correct side of it—a substrate that constrains emission without distorting cognition and searches trajectories speculatively; a cognition layer that spends compute by its value under a co-evolved verifier; a memory layer that consolidates while idle; an evolution layer that improves itself under a gate we prove non-degrading even under adaptive, continual operation; a governance layer with formal runtime guarantees; and an amplification layer that learns from a frontier teacher and from a privacy-preserving federation of peers. We proved geometric cost decay, federated acceleration under a bounded privacy budget, near-optimal compute allocation, and an honest weak-to-strong ceiling. The result is a falsifiable program—and a buildable product—for a local agent that

specializes, privately and permanently, until on the distribution that matters to its user it surpasses a frozen frontier model, and that, turned upward and outward, makes both the frontier model and the whole community of users continually better. Judgment belongs where its inputs are: below the wall, on your own machine, compounding safely.

Appendix A

Appendices

A.1 Notation

Symbol	Meaning
c_t, o_t, a_t	context, observation, action at step t
π_θ, θ	local policy and parameters; θ^{fr} frozen frontier
$s_t, \mathcal{K}$	engine state; KV-cache (radix) tree
$V_\phi, \hat{V}, \mathcal{M}$	verifier; probe estimate; DP holdout mechanism
H, C, m	probe set, canary set, probe size
τ, α, β	gate margin; DP accuracy and failure probability
$(\epsilon_{\text{dp}}, \delta_{\text{dp}})$	differential-privacy parameters
$b_t, B, N(b)$	compute level, episode budget, branches at level b
λ_t	VoI shadow price of compute at step t
$\Delta \hat{V}(u), \kappa(u)$	estimated value and cost of compute action u
q_t, γ, δ_0	escalation rate, absorption, novelty/drift rate
$\delta_0^{\text{fed}}, \sigma, n$	federated novelty rate, transfer fraction, #instances
$C_{\text{loc}}, C_{\text{fr}}$	local and frontier per-step cost
η_{w2s}	weak-to-strong elicitation efficiency
$\mathcal{D}_u, \Phi$	user task distribution; full CRUCIBLE system around a model

A.2 The Control-Surface Contract

An inference-native runtime is defined by the orchestrator/engine interface. CRUCIBLE requires the engine to expose: (1) **logit access & masking**—read z_t , apply a grammar mask m_t before sampling; (2) **cache control**—read, fork, prune the KV-cache tree $\mathcal{K}$, query prefix-sharing; (3) **speculation hooks**—read draft proposals and acceptance, register side-effect-free pre-execution; (4) **verifier hooks**—per-step/per-trajectory scores from the co-evolved GenPRM/AgentPRM; (5) **reversible adaptation**—apply, stack, drop, merge ROSE adapters between episodes, checkpoint/restore; (6) **idle scheduling**—run STC under device-idle and power constraints; (7) **settlement boundary**—present each action and its declared effects to the governance monitor before commit. Any engine providing these surfaces (SGLang with extensions) can host CRUCIBLE; no API-bounded interface (Def. 3.1) provides any of them.

Algorithm 2 The corrected gate (offline, within an STC window)

Require: candidate edits $\{\delta_i\}$ (teacher-grounded EATS/FEATS abstractions with SGC curricula), DP holdout $\mathcal{M}$ over H , online-FDR state, canary C , margins τ, κ , window w

- 1: **for** each candidate δ_i **do**
- 2: $p_i \leftarrow \mathcal{M}.\text{MARGINTEST}(\theta, \delta_i, \tau; \text{SGC held-out variations})$
- 3: $\alpha_i \leftarrow \text{SAFFRON.THRESHOLD}()$
- 4: **if** $p_i \leq \alpha_i$ **and** δ_i passes governance monitors **then**
- 5: checkpoint; apply δ_i as a ROSE orthogonal adapter; $\text{SAFFRON.EARN}()$
- 6: **else**
- 7: discard δ_i
- 8: **end if**
- 9: **end for**
- 10: **if** $V(\theta; C) < V(\theta_{\text{ckpt}}; C) - \kappa$ over last w commits **then**
- 11: drop adapters to last C -passing checkpoint ▷ Prop. 6.7
- 12: **end if**
- 13: periodically merge/prune ROSE adapters (continual-merging)

Algorithm 3 One FEATS federated round (privacy-preserving)

Require: local gate-passed abstractions A_{loc} , DP budget $(\epsilon_{\text{dp}}, \delta_{\text{dp}})$, aggregator

- 1: redact/hash symbolic parts; add calibrated DP noise to numeric parts [50, 49]
- 2: upload only privatized $\tilde{A}_{\text{loc}}$ to the aggregator
- 3: receive consolidated bundle $\bar{A}$ (averaged adapters, deduped skills)
- 4: **for** each incoming $\delta \in \bar{A}$ **do**
- 5: run $\text{CORRECTEDGATE}(\delta)$ locally ▷ Alg. 2; never bypassed
- 6: **end for**
- 7: update cumulative privacy loss by composition ▷ Thm. 6.9

A.3 Extended Algorithms

A.4 Attack Catalog

- A. Prompt injection via tool output.** *Defense:* capability sandboxing denies the authority to exfiltrate; the monitor blocks out-of-workspace writes and non-allowlisted egress before settlement (Chapter 4.7).
- B. Tool capability overreach.** *Defense:* the capability schema bounds scopes; out-of-scope effects are refused at the Atomix boundary (§4.2.3).
- C. Speculative side-effect leakage.** *Defense:* only side-effect-free actions pre-execute; others stage behind a frontier with compensation handlers.
- D. Reward hacking of the gate.** *Defense:* teacher-grounded labels, a co-evolved/execution-grounded verifier, hardening, and a constitution probe in the canary [17, 19, 28].
- E. Validation overfitting.** *Defense:* DP reusable holdout with a computable horizon and scheduled refresh (Thm. 6.5, Cor. 6.15).
- F. Catastrophic self-edit.** *Defense:* orthogonal adapters bound interference; canary detects regression; reversibility drops the adapter (ROSE).
- G. Federated poisoning.** *Defense:* recipients re-gate every incoming abstraction; a non-improving contribution is rejected (Thm. 6.9).

- H. Federated privacy inference.** *Defense:* only DP-noised/redacted abstractions are shared; cumulative loss bounded by composition; activations never shared.
- I. Memory poisoning.** *Defense:* consolidation runs through the same gate and monitors; PANDO confidence demotion removes skills that later fail.

A.5 Design Alternatives Considered and Rejected

- Whole-trajectory grammar.** Rejected: constrains cognition, degrading reasoning [6]. Chosen: phase-scoped grammar (§4.2.1).
- Alpha-investing alone for the gate.** Rejected: controls mFDR, not non-degradation (Prop. 6.8). Chosen: DP holdout + FDR + reversibility.
- Full-finetune self-edits.** Rejected: forgetting and irreversibility. Chosen: ROSE.
- Majority-vote unsupervised TTRL as the core signal.** Rejected: invites spurious-reward hacking [26]. Chosen: teacher-grounded EATS.
- Continuous latent reasoning.** Rejected: shortcut exploitation, not deduction [29]. Chosen: free-text cognition with adaptive compute.
- Cross-tenant Petals swarm.** Rejected: activation-inversion privacy breach [52]. Chosen: trusted-domain pooling and DP federation.
- Static confidence-threshold cascade.** Rejected: leaves savings on the table. Chosen: learned preference router [46].
- Natural-language guardrails.** Rejected: no guarantees. Chosen: compiled temporal-logic monitors [43, 44].

A.6 Glossary of CRUCIBLE Mechanisms

- EATS** Escalation-as-Training-Signal: distill each frontier escalation into gated, teacher-grounded local improvement (§5.1).
- FEATS** Federated EaTS: privacy-preserving collective improvement via DP, gate-passed abstractions (§5.1.6).
- VGSS** Verifier-Guided Speculative Search: bounded trajectory tree search, verifier-pruned, on a shared cache (§5.2.1).
- VOI** Value-of-Information compute market: per-step allocation of compute by value per FLOP (§5.2.2).
- STC** Sleep-Time Consolidation: idle-window pre-computation, consolidation, gated distillation, and adapter merging (§5.2.3).
- ROSE** Reversible Orthogonal Self-Edits: weight edits as orthogonal, separable, mergeable low-rank adapters (§5.2.4).
- SGC** Self-Generated Curriculum: expand each escalation into held-out variations so distillation generalizes (§5.1.4).

Conclusion and Future Research

CRUCIBLE argues that the decisive frontier for local agents is not only model scale but runtime architecture. The book’s central contribution is a full-stack account of how a small local model can become a persistent, private, improving agent when the agent loop crosses the API-boundary wall and operates on inference-native control surfaces.

The primary findings are architectural, theoretical, and methodological. Architecturally, the six-layer stack identifies the control surfaces that must be preserved. Theoretically, the gate, cost-decay, federation, and weak-to-strong analyses state when improvement can be expected and when it cannot. Methodologically, the evaluation protocol requires component evidence, longitudinal evidence, privacy accounting, adversarial tests, and a fair scaffolded-frontier baseline.

Future research should build independent implementations of the control-surface contract, measure the long-term stability of gated self-improvement, test federation under realistic privacy budgets, and develop contamination-resistant benchmarks for local agent specialization. The strongest version of the claim will not be established by a single leaderboard result, but by repeated longitudinal studies showing safe, private, cost-effective improvement on real user distributions.

Publishing Proposal and Peer-Review Plan

A.7 Pitch

This book introduces CRUCIBLE, an inference-native runtime for local small-language-model agents. It argues that many frontier-agent capabilities depend less on parameter count than on architecture: cache reuse, structured emission, speculative search, verifier feedback, memory, gated self-improvement, runtime governance, and privacy-preserving federation. The book offers a blueprint for turning those mechanisms into a coherent research program and buildable product.

A.8 Target Audience

The primary audience is researchers and engineers working on language-model agents, local AI systems, inference engines, AI safety, privacy-preserving learning, and agent evaluation. A secondary audience includes technical founders, research labs, and publishers interested in local-first AI infrastructure.

A.9 Market and Contribution

The manuscript occupies the space between a systems monograph, an AI-safety treatise, and a product-oriented architecture book. Its distinctive claim is that local agents should be evaluated as persistent systems rather than as stateless prompt wrappers.

A.10 Proposed Peer Review

External review should include at least one expert in inference systems, one in agent evaluation, one in privacy or federated learning, and one in AI safety or runtime verification. Reviewers should be asked to evaluate novelty, internal consistency, implementation plausibility, formal assumptions, and falsifiability.

A.11 Contract Materials

A publisher-facing proposal should include a synopsis, annotated chapter outline, target readership, competing titles, sample chapters, author biography, schedule, figures list, and a statement of permissions for reproduced or adapted material.

Bibliography

- [1] P. Belcak *et al.*, “Small Language Models are the Future of Agentic AI,” arXiv:2506.02153, 2025. <https://arxiv.org/abs/2506.02153>
- [2] R. Liu *et al.*, “Can 1B LLM Surpass 405B LLM? Rethinking Compute-Optimal Test-Time Scaling,” arXiv:2502.06703, 2025. <https://arxiv.org/abs/2502.06703>
- [3] C. Snell *et al.*, “Scaling LLM Test-Time Compute Optimally...,” arXiv:2408.03314, 2024. <https://arxiv.org/abs/2408.03314>
- [4] J. Wang *et al.*, “Mixture-of-Agents Enhances LLM Capabilities,” arXiv:2406.04692, 2024. <https://arxiv.org/abs/2406.04692>
- [5] “XGrammar-2: Efficient Dynamic Structured Generation Engine for Agentic LLMs,” arXiv:2601.04426, 2026. <https://arxiv.org/abs/2601.04426>
- [6] Z. R. Tam *et al.*, “Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of LLMs,” EMNLP 2024 (Industry); arXiv:2408.02442. <https://arxiv.org/abs/2408.02442>
- [7] A. Cooper, “A Guide to Structured Generation Using Constrained Decoding,” 2024. <https://www.aidancooper.co.uk/constrained-decoding/>
- [8] Y. Li *et al.*, “EAGLE-3: Scaling up Inference Acceleration of LLMs via Training-Time Test,” arXiv:2503.01840, 2025. <https://arxiv.org/abs/2503.01840>
- [9] “Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows,” arXiv:2602.14849, 2026. <https://arxiv.org/abs/2602.14849>
- [10] A. Zhou *et al.*, “Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models,” ICML 2024; arXiv:2310.04406. <https://arxiv.org/abs/2310.04406>
- [11] L. Zheng *et al.*, “SGLang: Efficient Execution of Structured Language Model Programs” (RadixAttention), arXiv:2312.07104, 2023. <https://arxiv.org/abs/2312.07104>
- [12] “HybridFlow (veRL): A Flexible and Efficient RL Post-Training Framework,” 2024. <https://github.com/volcengine/verl>
- [13] B. Jin *et al.*, “Search-R1: Training LLMs to Reason and Leverage Search Engines with RL,” 2025. <https://github.com/PeterGriffinJin/Search-R1>
- [14] L. Zhang *et al.*, “Generative Verifiers: Reward Modeling as Next-Token Prediction,” arXiv:2408.15240, 2024. <https://arxiv.org/abs/2408.15240>
- [15] “GenPRM: Scaling Test-Time Compute of Process Reward Models via Generative Reasoning,” arXiv:2504.00891, 2025. <https://arxiv.org/abs/2504.00891>
- [16] “AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress,” Proc. ACM Web Conference, 2026. <https://dl.acm.org/doi/10.1145/3774904.3792551>
- [17] “Cooper: Co-Optimizing Policy and Reward Models in RL for LLMs,” arXiv:2508.05613, 2025. <https://arxiv.org/abs/2508.05613>

- [18] “Adversarial Training of Reward Models,” arXiv:2504.06141, 2025. <https://arxiv.org/abs/2504.06141>
- [19] “LLMs Gaming Verifiers” (isomorphic vs. extensional verification), ICLR 2026 Workshop; arXiv:2604.15149. <https://arxiv.org/abs/2604.15149>
- [20] “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models,” arXiv:2510.04618, 2025. <https://arxiv.org/abs/2510.04618>
- [21] G. Wang *et al.*, “Voyager: An Open-Ended Embodied Agent with LLMs,” arXiv:2305.16291, 2023. <https://arxiv.org/abs/2305.16291>
- [22] Y. Li *et al.*, “PANDO: Efficient Multimodal AI Agents via Online Skill Distillation,” arXiv:2605.24785, 2026. <https://arxiv.org/abs/2605.24785>
- [23] “TTRL: Test-Time Reinforcement Learning,” arXiv:2504.16084, 2025. <https://arxiv.org/abs/2504.16084>
- [24] “CG-TTRL: Context-Guided Test-Time Reinforcement Learning for On-Device LLMs,” arXiv:2511.06430, 2025. <https://arxiv.org/abs/2511.06430>
- [25] A. Zweiger *et al.*, “Self-Adapting Language Models (SEAL),” arXiv:2506.10943, 2025. <https://arxiv.org/abs/2506.10943>
- [26] R. Shao *et al.*, “Spurious Rewards: Rethinking Training Signals in RLVR,” arXiv:2506.10947, 2025. <https://arxiv.org/abs/2506.10947>
- [27] “Spurious Rewards Paradox: Mechanistically Understanding How RLVR Activates Memorization Shortcuts,” arXiv:2601.11061, 2026. <https://arxiv.org/abs/2601.11061>
- [28] “Reward Hacking Benchmark: Measuring Exploits in LLM Agents with Tool Use,” arXiv:2605.02964, 2026. <https://arxiv.org/abs/2605.02964>
- [29] “Do Latent Tokens Think? A Causal and Adversarial Analysis of Chain-of-Continuous-Thought,” arXiv:2512.21711, 2025. <https://arxiv.org/abs/2512.21711>
- [30] “Seed Diffusion: A Large-Scale Diffusion Language Model with High-Speed Inference,” arXiv:2508.02193, 2025. <https://arxiv.org/abs/2508.02193>
- [31] “Learning to Parallel: Accelerating Diffusion LLMs via Learnable Parallel Decoding,” arXiv:2509.25188, 2025. <https://arxiv.org/abs/2509.25188>
- [32] A. Turner *et al.*, “Steering Language Models with Activation Engineering,” arXiv:2308.10248, 2023. <https://arxiv.org/abs/2308.10248>
- [33] “Small Vectors, Big Effects: A Mechanistic Study of RL-Induced Reasoning via Steering Vectors,” arXiv:2509.06608, 2025. <https://arxiv.org/abs/2509.06608>
- [34] S. Ma *et al.*, “The Era of 1-bit LLMs: All LLMs are in 1.58 Bits,” arXiv:2402.17764, 2024. <https://arxiv.org/abs/2402.17764>
- [35] “BitNet a4.8: 4-bit Activations for 1-bit LLMs,” arXiv:2411.04965, 2024. <https://arxiv.org/abs/2411.04965>
- [36] C. Packer *et al.*, “MemGPT: Towards LLMs as Operating Systems” (Letta), arXiv:2310.08560, 2023. <https://arxiv.org/abs/2310.08560>
- [37] “CraniMem: Cranial-Inspired Gated and Bounded Memory for Agentic Systems,” arXiv:2603.15642, 2026. <https://arxiv.org/abs/2603.15642>
- [38] K. Lin *et al.*, “Sleep-time Compute: Beyond Inference Scaling at Test-time,” arXiv:2504.13171, 2025. <https://arxiv.org/abs/2504.13171>
- [39] “Orthogonal Subspace Learning for Language Model Continual Learning (O-LoRA),” arXiv:2310.14152, 2023. <https://arxiv.org/abs/2310.14152>

- [40] “OPLoRA: Orthogonal Projection LoRA Prevents Catastrophic Forgetting,” arXiv:2510.13003, 2025. <https://arxiv.org/abs/2510.13003>
- [41] “Subspace Geometry Governs Catastrophic Forgetting in Low-Rank Adaptation,” arXiv:2603.02224, 2026. <https://arxiv.org/abs/2603.02224>
- [42] “Merge before Forget: Single-LoRA Continual Learning via Continual Merging,” arXiv:2512.23017, 2025. <https://arxiv.org/abs/2512.23017>
- [43] “Enforcing Temporal Constraints for LLM Agents,” arXiv:2512.23738, 2025. <https://arxiv.org/abs/2512.23738>
- [44] “ProbGuard: Probabilistic Runtime Monitoring for LLM Agent Safety,” arXiv:2508.00500, 2025. <https://arxiv.org/abs/2508.00500>
- [45] “Causal Past Logic for Runtime Verification of Distributed LLM Agent Workflows,” arXiv:2605.20923, 2026. <https://arxiv.org/abs/2605.20923>
- [46] I. Ong *et al.*, “RouteLLM: Learning to Route LLMs with Preference Data,” arXiv:2406.18665, 2024. <https://arxiv.org/abs/2406.18665>
- [47] “A Unified Approach to Routing and Cascading for LLMs,” arXiv:2410.10347, 2024. <https://arxiv.org/abs/2410.10347>
- [48] “Federated In-Context LLM Agent Learning,” arXiv:2412.08054, 2024. <https://arxiv.org/abs/2412.08054>
- [49] “LanFL: Differentially Private Federated Learning with LLMs using Synthetic Samples,” arXiv:2410.19114, 2024. <https://arxiv.org/abs/2410.19114>
- [50] “Differentially Private Low-Rank Adaptation of LLMs using Federated Learning (DP-LoRA),” arXiv:2312.17493, 2023. <https://arxiv.org/abs/2312.17493>
- [51] “Prompt Inference Attack on Distributed LLM Inference Frameworks,” arXiv:2503.09291, 2025. <https://arxiv.org/abs/2503.09291>
- [52] “Prompt Inversion Attack against Collaborative Inference of LLMs,” arXiv:2503.09022, 2025. <https://arxiv.org/abs/2503.09022>
- [53] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, A. Roth, “The reusable holdout: Preserving validity in adaptive data analysis,” *Science* 349(6248), 2015; arXiv:1506.02629. <https://arxiv.org/abs/1506.02629>
- [54] R. Bassily *et al.*, “Algorithmic Stability for Adaptive Data Analysis,” STOC 2016; arXiv:1511.02513. <https://arxiv.org/abs/1511.02513>
- [55] A. Ramdas *et al.*, “SAFFRON: an Adaptive Algorithm for Online Control of the False Discovery Rate,” ICML 2018; arXiv:1802.09098. <https://arxiv.org/abs/1802.09098>
- [56] A. Fisher *et al.*, “Online FDR control for LORD++ and SAFFRON under positive, local dependence,” *Biometrical Journal*, 2024; arXiv:2110.08161. <https://arxiv.org/abs/2110.08161>
- [57] C. Burns *et al.*, “Weak-to-Strong Generalization,” arXiv:2312.09390, 2023. <https://arxiv.org/abs/2312.09390>
- [58] “EnsemW2S: Enhancing Weak-to-Strong Generalization with LLM Ensembles,” arXiv:2505.21959, 2025. <https://arxiv.org/abs/2505.21959>
- [59] S. Xu *et al.*, “Chain of Draft: Thinking Faster by Writing Less,” arXiv:2502.18600, 2025. <https://arxiv.org/abs/2502.18600>
- [60] T. Han *et al.*, “Token-Budget-Aware LLM Reasoning,” arXiv:2412.18547, 2024. <https://arxiv.org/abs/2412.18547>

- [61] “DroidSpeak: KV Cache Sharing for Cross-LLM Communication and Multi-LLM Serving,” arXiv:2411.02820, 2024. <https://arxiv.org/abs/2411.02820>
- [62] “KVComm: Enabling Efficient LLM Communication through Selective KV Sharing,” arXiv:2510.03346, 2025. <https://arxiv.org/abs/2510.03346>
- [63] “Let Models Speak Ciphers: Multiagent Debate through Embeddings (CIPHER),” arXiv:2310.06272, 2023. <https://arxiv.org/abs/2310.06272>
- [64] “ToolDec: Syntax Error-Free and Generalizable Tool Use for LLMs via Finite-State Decoding,” arXiv:2310.07075, 2023. <https://arxiv.org/abs/2310.07075>
- [65] “From Hallucination to Structure Snowballing: The Alignment Tax of Constrained Decoding in LLM Reflection,” arXiv:2604.06066, 2026. <https://arxiv.org/abs/2604.06066>
- [66] “Thinking Before Constraining: A Unified Decoding Framework for Large Language Models (In-Writing),” arXiv:2601.07525, 2026. <https://arxiv.org/abs/2601.07525>
- [67] J. Kim *et al.*, “Scaling Test-Time Compute for Agentic Coding” (Recursive Tournament Voting; Parallel-Distill-Refine), arXiv:2604.16529, 2026. <https://arxiv.org/abs/2604.16529>
- [68] “Agent-RLVR: Training Software Engineering Agents via Guidance and Environment Rewards,” arXiv:2506.11425, 2025. <https://arxiv.org/abs/2506.11425>
- [69] “FedASK: Differentially Private Federated Low-Rank Adaptation Beyond Fixed-Matrix (Double Sketching),” arXiv:2507.09990, 2025. <https://arxiv.org/abs/2507.09990>
- [70] A. Agache *et al.*, “Firecracker: Lightweight Virtualization for Serverless Applications,” USENIX NSDI, 2020. <https://www.usenix.org/conference/nsdi20/presentation/agache>
- [71] “Harness-Bench / From Model Scaling to System Scaling: scaffolding variance exceeds model variance,” arXiv:2605.27922, 2026. <https://arxiv.org/abs/2605.27922>
- [72] “Does SWE-Bench-Verified Test Agent Ability or Model Memory? (The SWE-Bench Illusion),” arXiv:2512.10218, 2025. <https://arxiv.org/abs/2512.10218>
- [73] “SWE-bench Pro: Contamination-Resistant Software-Engineering Agent Evaluation,” arXiv:2509.16941, 2025. <https://arxiv.org/abs/2509.16941>
- [74] “Activated LoRA: Fine-Tuned LLMs for Intrinsic (KV-cache reuse under adapters),” arXiv:2504.12397, 2025. <https://arxiv.org/abs/2504.12397>
- [75] “Efficiently Aligning Draft Models via Parameter- and Data-Efficient Adaptation (draft/target transfer gap),” arXiv:2603.09527, 2026. <https://arxiv.org/abs/2603.09527>
- [76] “Zombie Agents: Self-Reinforcing Prompt Injection in Self-Evolving Agents,” arXiv:2602.15654, 2026. <https://arxiv.org/abs/2602.15654>
- [77] “Malice in Agentland: Environment-Injected Prompt Injection into Agent Fine-Tuning Data,” arXiv:2510.05159, 2025. <https://arxiv.org/abs/2510.05159>
- [78] “BadSkill: Backdooring Agent Skill Libraries,” arXiv:2604.09378, 2026. <https://arxiv.org/abs/2604.09378>
- [79] “The Geometry of Alignment Collapse: When Fine-Tuning Breaks Safety,” arXiv:2602.15799, 2026. <https://arxiv.org/abs/2602.15799>
- [80] “AlignGuard-LoRA: Safety-Subspace-Preserving Low-Rank Adaptation,” arXiv:2508.02079, 2025. <https://arxiv.org/abs/2508.02079>
- [81] “Safe-LoRA: Preserving Safety Alignment in LoRA Fine-Tuning,” arXiv:2405.16833, 2024. <https://arxiv.org/abs/2405.16833>

- [82] “Wider or Deeper? Scaling Test-Time Search under Imperfect Verifiers,” arXiv:2503.04412, 2025. <https://arxiv.org/abs/2503.04412>
- [83] “DeFT: Decoding with Flash Tree-Attention (IO-aware tree attention),” arXiv:2404.00242, 2024. <https://arxiv.org/abs/2404.00242>
- [84] “ProjRes: Membership Inference on Federated LoRA Adapters,” arXiv:2604.21197, 2026. <https://arxiv.org/abs/2604.21197>
- [85] “RerouteGuard: Adversarial Rerouting of LLM Routers,” arXiv:2601.21380, 2026. <https://arxiv.org/abs/2601.21380>
- [86] “The 2026 Local-LLM Landscape: MoE Models (Qwen3-class, Gemma-class) at Frontier-Adjacent Quality on Consumer GPUs,” industry survey, 2026. <https://www.sitepoint.com/local-vs-cloud-ai-coding-performance-analysis-2026/>

Index

adaptive non-degradation, [25](#)
API-boundary wall, [1](#)
appendices, [44](#)
attack catalog, [44](#)

benchmarks, [29](#)

conclusion, [47](#)
control-surface contract, [44](#)
cost decay, [25](#)

EaTS, [19](#)
evaluation protocol, [3](#)

falsifiability, [3](#)
FEaTS, [19](#)
future research, [47](#)

implementation, [29](#)
inference-native orchestration, [11](#)

KV cache, [11](#)

literature review, [1](#)
local agents, [5](#)

methodology, [3](#)

peer review, [29](#), [48](#)
publishing proposal, [48](#)

research gap, [1](#)
ROSE, [19](#)

SGC, [19](#)
six-layer architecture, [11](#)
small language models, [5](#)
STC, [19](#)

thesis, [5](#)

VGSS, [19](#)
VoI, [19](#)

weak-to-strong ceiling, [25](#)